# Plagiarism Detection in Source Code using Machine Learning

By

**Sumisha Surendran**

A Thesis
Submitted to the Faculty of Graduate Studies
through the School of Computer Science
in Partial Fulfillment of the Requirements for
the Degree of Master of Science
at the University of Windsor

Windsor, Ontario, Canada

2024

©2024 Sumisha Surendran

Plagiarism Detection in Source Code using Machine Learning

by

Sumisha Surendran

APPROVED BY:

G. Zhang
Department of Mechanical, Automotive, and Materials Engineering

H. Fani
School of Computer Science

D. Wu, Advisor
School of Computer Science

May 28, 2024

## DECLARATION OF ORIGINALITY

I hereby certify that I am the sole author of this thesis and that no part of this thesis has been published or submitted for publication.

I certify that, to the best of my knowledge, my thesis does not infringe upon anyone's copyright nor violate any proprietary rights and that any ideas, techniques, quotations, or any other material from the work of other people included in my thesis, published or otherwise, are fully acknowledged in accordance with the standard referencing practices. Furthermore, to the extent that I have included copyrighted material that surpasses the bounds of fair dealing within the meaning of the Canada Copyright Act, I certify that I have obtained a written permission from the copyright owner(s) to include such material(s) in my thesis and have included copies of such copyright clearances to my appendix.

I declare that this is a true copy of my thesis, including any final revisions, as approved by my thesis committee and the Graduate Studies office, and that this thesis has not been submitted for a higher degree to any other University or Institution.

ABSTRACT

This thesis presents the development and evaluation of an advanced plagiarism detection system designed to analyze source code. The system leverages a combination of Convolutional Neural Networks (CNN), Long Short-Term Memory networks (LSTM), and a Code Pre-trained Model (CodePTM) to detect various levels of plagiarism within C programming submissions. The goal was to improve the detection accuracy across different types of plagiarism, from simple copy-paste to more sophisticated structural and semantic plagiarism that is difficult to detect with traditional methods.

The methodology encompasses a comprehensive preprocessing stage that involves tokenization, generation of Abstract Syntax Trees (AST), and transformation into vectors that encapsulate lexical, structural, and semantic features of the code. These features are then analyzed through a multistage feature extraction process, utilizing the strengths of CNNs to discern structural patterns, LSTMs to capture contextual dependencies, and CodePTMs to understand deep semantic relationships.

Our results demonstrate that the integrated approach significantly enhances the capability to detect a wide range of plagiarism activities, achieving high precision and recall rates across six defined levels of plagiarism. Specifically, the system excels in identifying complex plagiarism cases that involve significant modifications to the code's structure or semantics, which are traditionally the most challenging to detect.

The implications of this research are profound, offering not only a more effective tool for academic settings to uphold integrity and originality in programming assignments but also potential applications in legal and professional domains where code originality is critical. Future work can extend this model to other programming languages and explore the integration of real-time detection capabilities, further broadening the impact and applicability of this research.

# DEDICATION

To my family, whose endless love and sacrifice have been the foundation of my success.

ACKNOWLEDGEMENTS

I sincerely express my most profound gratitude towards my supervisor, Dr. Dan Wu, for all the insightful guidance, unwavering support, and constructive criticism throughout the development of this thesis.

I thank my thesis committee members, whose expertise has significantly contributed to my scholarly understanding and growth. Their invaluable feedback has been essential in refining my work. Special thanks to the School of Computer Science for providing the resources crucial for this research.

Finally, I wish to acknowledge all the authors and researchers whose work has laid the groundwork for this study. Their pioneering efforts have been the guiding stars of this academic endeavor.

TABLE OF CONTENTS

**DECLARATION OF ORIGINALITY** **III**

**ABSTRACT** **IV**

**DEDICATION** **V**

**ACKNOWLEDGEMENTS** **VI**

**LIST OF FIGURES** **IX**

**1 Introduction** **1**
1.1 Existing Solutions . . . . . 3
1.2 Thesis Motivation . . . . . 6
1.3 Problem Statement and Proposed Solution . . . . . 7
1.4 Thesis Contribution . . . . . 9
1.5 Thesis Outline . . . . . 9

**2 Related Works** **11**
2.1 Historical Overview of Plagiarism Detection Solutions . . . . . 11
2.2 Machine Learning in Plagiarism Detection . . . . . 14
2.3 Key Influential Studies . . . . . 15
2.3.1 Source Code Plagiarism Detection: A Machine Intelligence Approach [21] . . . . . 16
2.3.2 Source Code Plagiarism Detection Based on Abstract Syntax Tree Fingerprintings [56] . . . . . 17
2.3.3 Source Code Plagiarism Detection with Pre-Trained Model Embeddings and Automated Machine Learning[20] . . . . . 18
2.4 Current Challenges in ML-Based Detection . . . . . 19

**3 Relevant Background** **21**
3.1 Overview of Machine Learning . . . . . 21
3.2 Deep Learning in Plagiarism Detection . . . . . 23
3.2.1 Convolutional Neural Networks (CNNs) . . . . . 23
3.2.2 Recurrent Neural Networks (RNNs) . . . . . 27
3.2.3 Long Short-Term Memory (LSTM) Networks . . . . . 29
3.3 Code Pre-trained Models (CodePTMs) . . . . . 30

**4 Proposed Methodology** **32**
4.1 Data Preprocessing . . . . . 34
4.2 Feature Extraction . . . . . 42
4.3 Training and Optimization . . . . . 48
4.4 Output and Decision Making . . . . . 50

**5 Experiments and Results** **52**
5.1 Experimental Setup . . . . . 52
5.2 Evaluation Metrics and Computational Results . . . . . 53
5.3 Comparing Results . . . . . 62

**6 Conclusion and Future Work** **64**
6.1 Propositions for Future Endeavors . . . . . 65
6.2 Concluding Reflections . . . . . 66

**REFERENCES** **67**

**VITA AUCTORIS** **74**

LIST OF FIGURES


1 Levels of Source Code Plagiarism . . . . . 2

2 CNN architecture . . . . . 24

3 RNN Architecture . . . . . 28

4 Proposed Methodology . . . . . 33

5 Sample C program and randomly masked version . . . . . 36

6 Distribution of Plagiarism Complexity Levels in our Dataset . . . . . 37

7 Sample C program and generated tokens . . . . . 39

8 Example of token embeddings . . . . . 40

9 Sample C program and generated AST . . . . . 41

10 AST in a matrix representation . . . . . 42

11 Dataset Split . . . . . 49

12 Performance comparison of our proposed model with integrated and single-method feature vectors . . . . . 55

13 AUC-ROC curve . . . . . 56

14 Confusion matrix . . . . . 57

15 Confusion matrix by levels . . . . . 58

16 A case of structural and semantic obfuscation detected by our model 60

17 Control Structure and Logical Changes detected by our model . . . . 60

18 Case of Algorithmic Changes detected by our model . . . . . 61

19 Advanced Structural and Semantic Obfuscation case detected by our model . . . . . 61

20 Comparative analysis . . . . . 62

# CHAPTER 1

# *Introduction*

Plagiarism, as defined in the University of Windsor's student code of conduct, is the presentation of another's ideas, words, or artistic, scientific, or technical work as one's creation. When applied to source code, this definition includes the unauthorized use or close imitation of the logic, structure, etc., of someone else's code without proper acknowledgment[1]. In the scenario of programming assignments, this can range from copying entire blocks of code to the subtle replication of algorithms and functionalities[2]. This practice undermines academic integrity and devalues the educational process in computer science and related disciplines [1], [2]. Programming code is a unique form of intellectual creation combining language, logic, and design elements. Plagiarism in this context, therefore, goes beyond the mere replication of text—it can manifest in the theft of creative ideas, algorithms, and even structural designs [1], [3]. As we delve into the technical analysis of plagiarism in code, it is essential to understand that this violation can occur in both the visible text of the code and its underlying logical constructs [3]. To fully grasp the extent of source code plagiarism, it is crucial to recognize its levels. These levels range from blatant textual copying to sophisticated algorithmic mimics that may not be immediately evident [2], [3], [4]. Fig 1 and the list below explain the various levels of plagiarism in source code [4], [6], [8]. The explanation column in Fig 1 also provides a high-level explanation of the techniques used to detect each level of plagiarism. Let us discuss these levels in a bit more detail below :

- Level 1 - Textual Replication: The most blatant form of plagiarism is the direct copy-paste of code, which can be detected through simple textual comparison

| Level of Plagiarism | Explanation |
|---|---|
| Level 1: Textual Replication | Direct copying of code without any changes, easily detectable by text comparison techniques. |
| Level 2: Syntactic Alteration | Superficial modifications to the code such as renaming identifiers, changing comments, or reformatting, while the underlying code remains largely unchanged. Token-based analysis is typically used to detect these changes. |
| Level 3: Structural Modification | Subtle yet substantial changes to the structure of the code, like altering loops and conditionals, which maintain the original logic but alter the execution flow. Detecting this level often involves AST comparisons. |
| Level 4: Control Flow Transformation | Alterations to the sequence or organization of code blocks, such as changing the order of independent code segments or substituting equivalent control structures. This is detected through control flow graph analysis. |
| Level 5: Algorithmic Paraphrasing | Rewriting the underlying algorithm to achieve the same functionality in a non-obvious manner, making detection more complex and often requiring semantic analysis. |
| Level 6: Design-Level Mimicry | Copying the overall design or problem-solving approach of a program. This sophisticated level of plagiarism is the most difficult to detect and often requires an in-depth understanding of the program's domain and intent. |

Fig. 1: Levels of Source Code Plagiarism

[3],[4]. Algorithms that perform such comparisons, like string matching algorithms, are foundational to identifying this level of plagiarism[4]. Here, we consider direct textual similarity as the primary indicator of plagiarism [4].

- Level 2 - Syntactic Alteration: As plagiarists often disguise copying through cosmetic changes, altering variable names, or modifying comments, detection requires analysis of the code's syntax [3], [4]. This involves breaking down the code into its fundamental syntactic elements or tokens. We term this "token-based comparison," which allows plagiarism detection despite superficial changes [4].

- Level 3 - Structural Modification: At this level, plagiarism may involve changes to the structure of the code, such as modifying loops. Despite these changes, the core logic remains unchanged[8]. Detecting such modifications requires a more in-depth analysis, which can include examining the patterns and structures

within the code, such as its abstract syntax trees (ASTs) [8], [7].

- Level 4 - Control Flow Transformation: A more complex type of plagiarism involves altering the flow of the program, for example, changing the sequence or organization of code blocks [4]. Methods and tools for detecting this level of plagiarism analyze the 'control flow' of the program, which represents the order in which individual statements or blocks of code are executed [4], [8].

- Level 5 - Algorithmic Paraphrasing: Here, the plagiarist rewrites the algorithm in a way that performs the same function but is not directly recognizable[8]. It is necessary to employ methods to interpret the code's semantic meaning and purpose to detect such sophisticated plagiarism [4],[8].

- Level 6 - Design-Level Mimicry: The most subtle and challenging form of plagiarism to detect is where an entire design or solution approach is copied without directly using the same code [4], [5], [8]. It requires understanding the design principles and detecting parallels in the problem-solving approach [8].

Having delineated the varying levels of source code plagiarism, it becomes crucial to explore how these challenges have historically been addressed. The evolution of plagiarism detection has been shaped by the need to adapt to increasingly sophisticated methods of code appropriation [11]. The following section will explore the existing solutions to combat plagiarism in the academic communities. This exploration will highlight the technological advancements in detection tools and underscore the inherent limitations that current methodologies face, setting the stage for the motivation behind this thesis.

## 1.1 Existing Solutions

As we transition from understanding the diverse levels of source code plagiarism, we must examine the solutions developed over time to tackle this pervasive issue. This section discusses some of the methods and tools that have been instrumental in detecting plagiarism in programming assignments [8], [11], [12], [14], [15], [17]. The earliest

attempts to detect source code plagiarism were rudimentary and primarily manual [11]. Instructors would visually inspect student submissions for similarities, a time-intensive method prone to errors [11]. As computing classes grew and assignments became more complex, the need for automated tools became apparent [11], [12]. One of the first automated approaches to detecting source code plagiarism involved simple text-based comparison tools [14]. These tools, such as 'diff' in Unix, compare code files line by line, highlighting direct textual similarities [14]. However, their utility is limited to detecting Level 1 plagiarism—direct copying without any modifications [14]. As plagiarists became more sophisticated, employing Level 2 and Level 3 plagiarism techniques like renaming variables or changing the code structure slightly, detection methods evolved [14], [15]. Token-based systems emerged, parsing the code into tokens (basic syntactic units) rather than comparing raw text [8], [12], [14]. These systems are more robust against simple modifications and can identify code altered superficially [15]. One of the very famous tools, MOSS, uses token-based systems [11]. It Analyzes programs for similar patterns by tokenizing the text and comparing the resulting tokens. While effective at identifying similar structures, MOSS struggles with high-level algorithmic paraphrasing and design-level mimics(Levels 4, 5, and 6) [11], [12], [13], [14]. AST-based methods were then introduced to address structural and some levels of logical plagiarism (Levels 3 and 4) [7], [8]. AST-based methods analyze the abstract syntax tree of a program [7], [8], [12]. An AST represents the syntactic structure of the code, abstracting away from the literal code written to focus on the structure formed by statements, loops, and other control elements [7], [8], [12], [14]. One such research that utilizes the AST-based method is "Plagiarism Detection System using AST Analysis"[17]. This research compares the structural properties of code by constructing and analyzing ASTs[17]. While capable of identifying structural similarities, these methods require more computational resources and still fall short in detecting semantic plagiarism[11],[14],[17]. To tackle higher levels of plagiarism, particularly algorithmic paraphrasing, newer research [21],[22],[24] has incorporated semantic analysis techniques[22]. These involve understanding the functionality and underlying intent of the code segments, going beyond mere structural

or syntactic similarities[21],[22]. Semantic code analysis employs machine learning techniques to understand the purpose and operation of code segments, comparing them based on the tasks they perform rather than how they are written[17]. Although they perform well, these methods have high complexity and computational demand[12],[17],[21],[22]. They also require significant training data and sophisticated model tuning to accurately differentiate between legitimately similar and plagiarized code [17]. But still, in recent years, advanced machine learning techniques such as CNNs have been used to address the complex challenges of plagiarism detection in textual and programming content[22],[23],[27]. These techniques have been employed to various extents, reflecting their potential to enhance the accuracy and depth of analysis in detecting plagiarized material[22],[23]. Convolutional Neural Networks (CNNs) are particularly valued for their ability to process and recognize patterns in image data, creatively applied to textual content by treating text segments as one-dimensional images. This approach allows CNNs to capture local and global patterns in syntax and formatting, which are crucial for identifying structural similarities in source code. For example, a study by Katta et.al in [2018] demonstrated how CNNs could effectively detect stylistic similarities in code that traditional text comparison tools might overlook[20]. While CNNs excel at capturing spatial dependencies within data, the sequential nature of source code, where the logic unfolds over lines and blocks, calls for a different approach[27]. This is where Recurrent Neural Networks (RNNs) come into play. RNNs, especially their LSTM variants, are tailored for analyzing data where order and context significantly influence the content's meaning, such as in programming languages. By processing code line by line, RNNs maintain an understanding of the code's evolving logic, making them exceptionally good at detecting plagiarism that involves reordering or modifying the code to disguise copying[27]. Although RNNs provide a robust framework for understanding the flow and structure of code, comprehending the intent and semantics requires a more nuanced analysis[27]. This is effectively handled by Natural Language Processing (NLP) techniques. NLP goes beyond the surface to probe the underlying meanings embedded in the code, especially within comments and documentation. This semantic layer is

crucial for distinguishing between codes that merely look similar and those that are genuinely derivative, thus refining the detection process to focus on intent as much as form[27],[28],[30].

While each method has progressively addressed more complex forms of plagiarism, some gaps still need to be addressed, particularly in detecting sophisticated logical and design-level plagiarism efficiently and accurately. The limitations of existing tools underscore the need for an integrated approach that combines various analytical techniques to capture a comprehensive spectrum of plagiarism types. This evolution from basic text comparison to sophisticated semantic analysis illustrates both the progress made and the challenges that persist in source code plagiarism detection[6],[7],[8],[10],[11],[12],[14],[17]. While these solutions have laid substantial groundwork, there remains an opportunity to enhance these technologies' integration and application, particularly in hybrid models that combine the strengths of each approach for a more comprehensive analysis.

## 1.2 Thesis Motivation

In addressing the limitations identified in existing plagiarism detection methodologies, this thesis recognizes the need for a system capable of comprehensively interpreting source code's syntax, structure, semantic content, and functional intent. Current tools often fall short when faced with sophisticated plagiarism tactics, such as semantic paraphrasing and structural mimicry, which are increasingly prevalent due to the advanced tools and resources available to students and professionals.

In this thesis, we establish a fundamental hypothesis concerning the superiority of our hybrid feature extraction technique, which integrates lexical, structural, and semantic analysis over traditional single-method approaches. This hypothesis underscores the anticipated enhancements in accuracy and robustness in detecting various levels of plagiarism within source code. This approach is grounded in developing

a unique feature vector that encapsulates lexical, structural, and semantic features. This vector is crucial in our proposed work because it allows our model to capture and analyze multi-dimensional aspects of code more effectively than methods that focus solely on one or two dimensions[13],[14],[16],[21],[22],[27]. By integrating these diverse features, the proposed system detects direct code similarities and uncovers deeper, more subtle instances of plagiarism, such as restructured logic or semantically similar code implemented in different syntactic forms.

This thesis does not claim to solve the complex problem of source code plagiarism completely. Instead, it humbly attempts to make a substantive contribution by advancing the technology available for detecting plagiarism. The motivations outlined here form the foundation upon which our research is built. The following sections will get into the formulation of the problem, the methodologies employed, and the expected contributions of this study to the field of plagiarism detection.

## 1.3 Problem Statement and Proposed Solution

The core objective of this thesis is to develop a machine-learning model capable of detecting plagiarism in source code, with a specific focus on C programs. This challenge is approached through a binary classification framework, wherein the model is trained to classify source code files as plagiarized or non-plagiarized. The problem can be precisely defined as:

$$f(X) = y$$

where:

- $X$ represents the input feature vector derived from pairs of source code files. This vector includes quantified attributes that encapsulate syntactic similarities, structural patterns, and semantic correlations critical in identifying instances of plagiarism. The development of these features is crucial, as it determines the model's ability to discern nuanced forms of code reuse that may not be overtly

apparent.

- $y$ is a binary label where $y = 1$ indicates that the pair of files is plagiarized, showing significant similarities that suggest potential unauthorized copying, and $y = 0$ indicates no plagiarism is detected.

- $f$ denotes the classification function, implemented by our proposed machine learning model, tasked with predicting the presence of plagiarism based on the analyzed features.

The proposed solution in this thesis involves the development of a hybrid feature extraction process that integrates Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Natural Language Processing (NLP) techniques. This approach captures the source code's structural and semantic intricacies. CNNs are usually utilized to analyze the structural patterns in code, such as indentation and syntax formatting, which indicate stylistic similarities[30],[31]. RNNs, particularly those employing Long Short-Term Memory (LSTM) units, are adept at processing sequences and are used to understand the logical flow and control structures within the code, making them ideal for identifying deeper, logical similarities that extend beyond mere textual matches[33]. Additionally, a pre-trained NLP model called CodePTM is used to grasp general language patterns and specific programming semantics, making it adept at interpreting source code[57]. This provides a layer of semantic analysis that helps distinguish between legitimately similar codes and potential plagiarism[32],[33]. This composite framework is expected to enhance plagiarism detection capabilities by assessing code through an innovative, integrated set of syntactic, semantic, and stylistic features. Chapter 4 will delve into the specifics of our proposed methodology, outlining the technical deployment of each model component and the rationale behind their integration. The effectiveness of this advanced approach will be evaluated using precision, recall, F1-score, and ROC-AUC metrics to confirm its accuracy and adaptability to evolving coding practices.

## 1.4 Thesis Contribution

With this thesis, we hope to contribute to the field of plagiarism detection by developing a machine-learning model that detects source code plagiarism up to level 5 with high recall and precision. While the underlying model follows a binary classification framework, the distinctive aspect of this research lies in its integration of CNNs, RNNs, and NLP for extracting features. This combination is crafted to capture a comprehensive spectrum of features from structural, sequential, and semantic perspectives, making it unique in the plagiarism detection field. Using NLP(Code Pre-trained Models (CodePTMs)), this research extends beyond existing syntactic analysis, enabling a deeper semantic understanding of the code. This aspect is critical in distinguishing between superficially similar code and genuinely plagiarized content, providing a nuanced analysis that previous models may lack. Additionally, this work incorporates a metadata analysis component, which examines contextual information surrounding the source code submissions, such as submission timestamps and coding patterns over time. These enhancements allow for a more comprehensive feature set that captures syntactic, structural, and semantic elements and temporal and contextual nuances often overlooked in other plagiarism detection systems.

While humbly recognizing the challenges inherent in completely eradicating plagiarism, this work can improve detection methodology by using a combination of feature vectors, which can help us detect plagiarism up to level 5. The contributions outlined, the results from our experiments, and a thorough discussion of their broader implications will be explored in Chapters 5 and 6, ensuring the clarity of the impact and relevance of the research within academic contexts.

## 1.5 Thesis Outline

The thesis is structured as follows:

- Chapter 2: Related Works – This chapter delves into the existing literature on

plagiarism detection methods, focusing on those employing machine learning techniques such as CNNs, RNNs, and NLP. It highlights current capabilities, identifies gaps, and justifies the need for the research proposed in this thesis.

- Chapter 3: Relevant Background – Provides information on the technologies and methodologies relevant to the proposed solution. This includes a discussion of CNNs, RNNs, and NLP techniques and their working principles.

- Chapter 4: Proposed Methodology – Describes the methodology developed to address the problem of detecting plagiarism in C programs. This includes the architecture of the hybrid machine learning model, feature engineering strategies, and the integration of CNNs, RNNs, and NLP for plagiarism detection.

- Chapter 5: Details the experimental setup, the data collection process, and the methods used to test the proposed model's efficacy. This chapter presents the results of the experiments, providing an analysis of the model's performance based on various metrics.

- Chapter 6:Concludes the thesis by summarizing the main findings and contributions. This chapter also discusses the limitations of the current study. It outlines potential future research directions that can build on the work done, suggesting improvements and exploring how the model might be adapted or extended.

To fully appreciate the proposed solution's contributions, it is important to understand the context of existing research and developments in plagiarism detection and machine learning applications. The next chapter, the literature review, will comprehensively analyze current methodologies, previous research, and theoretical underpinnings that inform this thesis. This will highlight the advancements made and underscore the necessity for the proposed improvements.

# CHAPTER 2

# *Related Works*

In this chapter, we examine the landscape of source code plagiarism detection, with particular emphasis on the evolution of detection methodologies and the increasing role of machine learning techniques in this domain. Our work is guided by a review of the academic literature spanning several decades of research and development. This exploration is vital for situating our study within the broader academic discourse, offering insights into the trajectory of technological advancements and pedagogical challenges associated with plagiarism in computer science education. We aim to contextualize our work amidst these efforts, elucidating the persistent challenges and the emergent strategies that have shaped current approaches to plagiarism detection.

## 2.1 Historical Overview of Plagiarism Detection Solutions

Source code plagiarism, a concern since the genesis of computer science education, has become increasingly complex due to digital resource proliferation and collaborative platforms [15]. Initially, educators tackled plagiarism manually, comparing student code submissions line by line, which was time-consuming and error-prone [13]. The demand for more sophisticated detection mechanisms grew with the evolution of programming languages and teaching methodologies. In the late 1960s, as computer science programs expanded, the rudimentary nature of plagiarism detection, based on instructors' memory of student work, led to the development of systematic methods in the following decades [13], [14], [16], [18], [21], [22], [23], [27], [31]. By the

1990s, the first automated tools for source code plagiarism detection emerged, revolutionizing the process. MOSS, developed by Alex Aiken, was an early automated system utilizing tokenization and hashing [21], [28]. Tokenization involves breaking down the source code into fundamental units or tokens, such as keywords, operators, and identifiers, to abstract away from stylistic variances like spacing and comments [28]. Hashing followed, which mapped these tokens to numerical values, creating a 'fingerprint' of the code to detect identical code blocks across different submissions efficiently. JPlag, appearing in the late 1990s, enhanced the tokenization process by considering not just the individual tokens but also their sequence, providing a more refined comparison and better detection of code that had been restructured or modified slightly [13], [14], [16], [18], [21], [22], [23], [27], [31]. The system was designed to process these token sequences through an internal algorithm, comparing the structure and order of tokens to find matching patterns [17], [21]. The early 2000s saw the incorporation of string-matching algorithms, such as the Running-Karp-Rabin Greedy String Tiling algorithm (RKR-GST). RKR-GST worked by identifying the longest substring common to both code sets, tiling them to prevent re-matching, and then iterating the process to find additional matches [21], [22], [23], [27], [31]. This method improved the granularity of comparison, allowing the system to detect plagiarized code even if it was rearranged or spread across different locations within the files. Simultaneously, graph-based approaches emerged, which involved translating the source code into a graph structure, where nodes represented code entities (e.g., variables, functions) and edges represented relationships (e.g., function calls, loops). These graphs were then analyzed for isomorphic patterns, enabling plagiarism detection despite syntax changes—valuable for identifying structural similarities that traditional text comparisons would miss [13], [14], [16], [18], [21], [22]. By the mid-2000s, the limitations of these purely algorithmic approaches in tackling advanced obfuscation techniques prompted the exploration of machine learning [21], [22], [23], [27], [31]. Early machine learning applications employed algorithms such as decision trees and k-nearest neighbors (KNN) [25], [26], [27]. Decision trees are operated by creating a tree-like model of decisions based on code features. At the same time,

KNN classified code samples by identifying similar training examples in feature space. Although a step forward, these methods hinged on labor-intensive feature engineering and needed more data quality [21], [22], [23], [27], [31]. The advent of deep learning in the 2010s brought a quantum leap in plagiarism detection capabilities. Convolutional Neural Networks (CNNs), renowned for image analysis, were adapted to analyze source code structure, interpreting code blocks as image-like data matrices to recognize patterns in formatting and layout. Recurrent Neural Networks (RNNs), particularly those with Long Short-Term Memory (LSTM) units, excelled at processing sequential information, such as the logical flow of code across various lines and functions, making them effective at identifying complex plagiarism cases that involved deep modifications [28], [29], [30], [32], [33]. The introduction of Natural Language Processing (NLP) techniques further extended the capabilities of plagiarism detection systems[33]. NLP algorithms process the natural language components of code, such as comments and documentation, using techniques like semantic embedding and syntax parsing to understand the intent and meaning behind the code, elevating the detection process to consider semantic and syntactic similarities [33], [34], [35]. While comments and documentation are critical for understanding the intent and purpose behind code segments, NLP can also analyze the code itself. This includes identifying patterns within the code that signify certain functionalities, such as loops for iteration or conditionals for decision-making processes.

Having traversed the timeline of traditional plagiarism detection techniques, from manual inspections to sophisticated algorithmic approaches, it becomes clear that the increasing complexity of plagiarism strategies has significantly influenced the evolution of these methods [13], [14], [21], [22], [23], [27], [28], [29], [30]. The limitations of purely algorithmic solutions, particularly their struggles with advanced obfuscation techniques and the growing sophistication of plagiarism methods, necessitated a shift towards more adaptable and intelligent systems. This transition marked the popularity of integrating machine learning (ML) into plagiarism detection, a pivotal development that has reshaped the landscape of plagiarism detection technologies.

The next section looks at machine-learning solutions developed to solve the plagiarism problem.

## 2.2 Machine Learning in Plagiarism Detection

Machine learning's initial forays into plagiarism detection utilized relatively simple classification models. Decision trees, for example, were used to classify segments of code based on features extracted from the code's syntax and structure. Similarly, k-nearest neighbors (KNN) algorithms classified code by analyzing the 'closeness' of its features to those of known examples of plagiarism. These methods, however, were challenging. They required extensive feature engineering to select, modify, and create features that improve the model's accuracy and were constrained by the quality and diversity of the training data available [28], [29], [30], [33].

As machine learning continued to evolve, so did its application in detecting source code plagiarism. The introduction of deep learning models, especially Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), in the late 2010s marked a significant advancement. CNNs, with their powerful capability to detect patterns and structures in visual data, were adapted to analyze the structural patterns of code as if they were images [31]. This allowed them to detect direct copying and structural similarities that might not be evident through text alone. With time, Recurrent Neural Networks (RNNs), known for their effectiveness in processing sequential data, became instrumental in understanding the logical flow of code [32]. This capability was crucial for identifying plagiarized code that had undergone significant modifications, such as reordering code blocks or control flow changes, which other methods often missed. Introducing Machine Learning techniques enhanced the accuracy of plagiarism detection systems and their adaptability to various programming languages and styles, addressing some of the most significant limitations of previous methods [30], [31], [32]. Although better than previous methods, the initial application of machine learning in plagiarism detection brought several challenges,

notably in feature selection and model training [30], [31], [32]. Feature selection involves deciding which aspects of the source code are most relevant and how to effectively encode these aspects for the machine learning models[28]. Early efforts required meticulous manual effort to determine the most indicative features of plagiarism, such as specific syntactic patterns, structural configurations, or stylistic elements. Moreover, the performance of these models heavily depended on the quantity and quality of the training data available[29]. In many cases, the available datasets needed to be sufficiently large or diverse to train robust models, leading to issues with overfitting where models performed well on training data but needed to generalize to new, unseen data sets [29], [30], [31]. To overcome these challenges, researchers began employing automated feature engineering techniques and exploring more sophisticated algorithms that could learn more complex patterns with less manual intervention [33]. For instance, unsupervised learning approaches, such as clustering, were experimented with to detect plagiarism without needing a labeled dataset [34]. Despite these advancements, machine learning-based plagiarism detection still faces a few challenges, particularly in handling the diversity of programming languages and the sophistication of plagiarism techniques. The adaptability of these systems across different coding styles and languages remains a critical issue, as does their ability to detect advanced plagiarism tactics involving semantic obfuscation or logical restructuring [34]. In the next section, let us look at a few research works done regarding detecting plagiarism in source code using machine learning techniques.

## 2.3 Key Influential Studies

This literature review section focuses on a few research works that have notably influenced our research in machine learning and source code plagiarism detection. These studies are foundational in their contributions and instrumental in highlighting the gaps and challenges our research aims to address.

Each selected study sheds light on different facets of plagiarism detection, from

algorithmic techniques to implementation challenges. While they mark significant strides in the field, they also leave open opportunities and areas needing further exploration. It is within these areas that our research finds its impetus.
We will briefly summarize these key studies and discuss how they have shaped our research approach.

### 2.3.1 Source Code Plagiarism Detection: A Machine Intelligence Approach [21]

This research discusses using machine learning and deep learning methods to detect plagiarism in C programming assignments. The paper explores various methodologies beyond text-based approaches, focusing on overcoming the challenges posed by code obfuscation techniques. This work utilized a dataset (obtained from Dataport), which is the same as ours. The features are extracted from the source codes using the MILEPOST GCC compiler, a machine learning-enabled tool[21]. This involves analyzing different aspects of the code, such as the number of basic blocks and control flow graph statistics. The feature sets train supervised learning models, including Support Vector Machines (SVM), Decision Trees, and K-nearest neighbors (KNN)[21]. An unsupervised algorithm(attention-based transformers) was tested to eliminate any kind of bias towards a dataset. The study reported precision, recall, and F1-score at 0.92, 0.83, and 0.87, respectively. These models aim to classify pairs of programs as plagiarized or non-plagiarized based on the extracted features. The results showed that machine learning methods provided superior accuracy in detecting level 1, 2, and 3 plagiarism compared to tools like MOSS and JPlag, particularly in handling obfuscated code[21].

The authors identify their future research direction in applying machine learning to combat higher levels of plagiarism. Their research publication highlights the challenges of current work in detecting plagiarism in code deliberately obfuscated(level 3 and above) to evade detection[21]. This encouraged us to find a solution that could

detect plagiarism beyond levels 1, 2, and 3.

### 2.3.2 Source Code Plagiarism Detection Based on Abstract Syntax Tree Fingerprintings [56]

This research discusses a novel approach to detecting plagiarism in source codes using the structure of Abstract Syntax Trees (AST) and fingerprinting techniques. The approach is designed to detect similarities in C programming language source codes[56]. According to this work, ASTs represent the logical structure of the code in a tree format, capturing both syntax and semantics. Source codes are parsed to generate their ASTs, which reflect the structural and logical organization of the code. Fingerprints of ASTs are generated by hashing the nodes of the trees[56]. These fingerprints are used to identify and compare similar nodes between different ASTs. Weights are assigned to the nodes of the AST based on their importance and contribution to the program's functionality. The comparison of ASTs involves matching nodes between trees based on their fingerprints[56]. This process identifies similar code structures across different programs. A similarity score is calculated based on the number of matched nodes[56]. This score quantifies the degree of similarity between two pieces of code.

This research work used the same dataset that we will be using for our research. The research can detect plagiarism for detecting levels 1, 2, and 3, has yielded an accuracy of 0.9, precision of 0.84, recall of 0.71, F1 of 0.63, false positive rate (FPR) of 4.15%, and false negative rate (FNR) of 28.33%[56]. The authors disclosed that the system faces challenges with highly obfuscated code and code that employs complex syntactic transformations above level 3, which the AST fingerprinting might not fully capture as the model does only structure-based detections[56].

### 2.3.3 Source Code Plagiarism Detection with Pre-Trained Model Embeddings and Automated Machine Learning[20]

This research addresses source code plagiarism in computer science education using machine learning techniques[20]. The study treats plagiarism detection as a binary classification problem, leveraging pre-trained model embeddings and Automated Machine Learning (AutoML) to enhance the detection process[20]. The dataset used in this work was created by the authors and contained 1989 C program files, with 322 of those labeled plagiarised. The methodology involves converting source codes into vector representations that capture code semantics, known as contextual embeddings. These embeddings are generated using pre-trained models tailored to the source code [20]. The cosine similarity scores from these embeddings are then used as features. The classifier selection and parameter tuning are performed using AutoML, which helps identify the most effective classifiers without manual intervention[20]. The approach uses multiple classification models and tests their effectiveness in representing the source code features for plagiarism detection. The research came up with a precision of 0.79, recall of 0.86, and F1 Score of 0.82 for detecting plagiarism at level 5 [20]. While the results are average for level 5, the document acknowledges potential limitations in ensuring the model accurately detects plagiarism across various coding styles and structures.

The authors disclosed that the model could detect changes in the algorithm that maintain the same functionality but alter the underlying logic, making it more complex, which is level 4, and that the model might detect some instances if the alterations impact semantic meaning significantly, which would be level 4. The authors pointed out that the model performed poorly in detecting levels 5 and 6 of plagiarism [20] and suggested it as a future direction.

## 2.4 Current Challenges in ML-Based Detection

Machine-learning-based plagiarism detection systems are among some of the advanced and effective tools available currently for plagiarism detection [15], [16], [20], [23], [29], [35], [36]. However, challenges persist, such as complex parsing of diverse coding languages and styles[29]. Machine learning models often struggle with syntactic and semantic variations across different programming languages[24]. For instance, the same logical functionality can be implemented in various ways, using different syntax in languages like Python, Java, or C++. This diversity poses a challenge in feature extraction and pattern recognition[26], [27]. Models may need to be more accurate across languages, leading to higher false negatives in lesser-represented languages[35]. The variance in coding styles (e.g., use of libraries and coding conventions) further complicates the model's ability to identify plagiarism consistently[31], [33]. Many of the discussed works and models have efficiently identified direct textual copying and some structural modifications. Yet, they need to improve when faced with more sophisticated forms of code obfuscation. Advanced obfuscation involves techniques like code restructuring, logic alteration, and synonym functions, which can significantly alter the code's appearance without changing functionality[33], [35]. Detecting such obfuscations requires deep semantic analysis and understanding code logic beyond superficial syntactic analysis[35]. Current feature extraction methods may not capture the essence of such alterations, leading to the model missing sophisticated instances of plagiarism. The primary reasons for these limitations are[15], [23], [29], [35], [36]:

- Dataset Constraints: Many studies have utilized relatively small datasets for training and testing their models. The lack of extensive and varied data can restrict the model's ability to learn and generalize complex patterns indicative of advanced plagiarism techniques.

- Detection Depth: Most models are adept at surface-level analysis—capturing lexical and syntactic similarities. However, they often lack the depth required to analyze the nuanced alterations that characterize higher levels(level 3 and above) of plagiarism, such as changes in the control flow or algorithmic para-

phrasing that retain the original program's functionality but not its form.

Despite advancements, the current literature review reveals an opportunity to improve detection capabilities, especially in handling advanced obfuscation techniques and diverse programming paradigms[33], [35]. Many tools still need to be improved in their ability to discern deeply altered yet functionally equivalent code[35]. Emerging trends[15], [16], [23], [27], [33], [35] indicate a move towards more AI-driven approaches. The future direction of plagiarism detection research points towards systems capable of learning from vast datasets, adapting to new plagiarism techniques, and offering more personalized and context-aware detection[35]. Some of the research works[5], [15], [16], [33] that we discussed above have leveraged the capabilities of machine learning and neural networks. While RNNs and CNNs show potential, particularly due to their ability to process sequences and hierarchical patterns in data, their combined application in plagiarism detection is still an emerging area of research that we see as an opportunity.

The literature reviewed allows us to develop a machine learning-based approach to source code plagiarism detection, which is the focus of this study. This work will combine RNN, CNN, and NLP approaches to enhance detection accuracy and adaptability while trying to detect plagiarism levels 1-5. This will be further discussed in detail in Chapter 4. Before moving to Chapter 4, the proposed methodology, we would like to discuss machine learning and the techniques we will use for this work. In Chapter 3, we will provide a high-level background on the technology, which will help us understand Chapter 4 better.

# CHAPTER 3

# *Relevant Background*

In this chapter, we aim to provide an overview of the foundational concepts and existing technologies that underpin our approach to detecting plagiarism in source code using machine learning techniques. Understanding and identifying plagiarism effectively upholds academic integrity and fosters fair educational practices and innovation in software development. As such, this chapter explores machine learning, convolutional neural networks, and recurrent neural networks. Understanding these areas is crucial for appreciating the complexity of the problem and the solution proposed in this thesis.

## 3.1 Overview of Machine Learning

Machine learning (ML) is a critical component of artificial intelligence that focuses on developing algorithms to learn from and make predictions or decisions based on data. In this section, we explore the fundamentals of machine learning, particularly emphasizing its applications in supervised learning, which is central to our approach to detecting plagiarism in source code. Machine learning algorithms are typically classified into three categories based on their learning style: supervised, unsupervised, and reinforcement learning [46].

- Supervised Learning: This type of learning uses a labeled dataset to train algorithms that can accurately predict the outcome for unseen data [46]. Each example in the training dataset includes an input object (typically a vector) and a desired output value (also called the supervisory signal). The supervised

learning algorithm analyzes the training data and produces an inferred function, which can be used for mapping new examples [46].

- Unsupervised Learning: In contrast, unsupervised learning involves training an algorithm on a dataset without labeled responses. Here, the goal is to infer the natural structure within a set of data points [46].

- Reinforcement Learning: Reinforcement learning is a type of learning where an agent learns to behave in an environment by performing actions and seeing the results [46]. It is not directly related to plagiarism detection but is an essential area within the scope of machine learning.

In the context of plagiarism detection, supervised learning is particularly relevant. The typical approach involves training a model on a dataset where each instance of code is labeled as 'plagiarized' or 'non-plagiarized.' This method effectively learns and predicts based on features extracted from the code, including syntactic patterns, coding style, logical structure, and even comments embedded within the code. The process of applying supervised learning in plagiarism detection can be broken down into several steps [46], [47]:

- Data Collection: Gather a comprehensive source code dataset, including plagiarized and non-plagiarized code examples.

- Feature Extraction: Develop algorithms to extract meaningful features from the code that are indicative of plagiarism. This might include metrics like code similarity, the frequency of specific tokens, structural patterns, etc.

- Model Selection: Choose appropriate machine learning models (e.g., decision trees, support vector machines, neural networks) that are capable of effectively learning from the features to distinguish between plagiarized and non-plagiarized code.

- Training: Train the model using a large subset of the collected data where the correct classification is known. This step involves adjusting the model parameters to minimize prediction errors based on the training data.

- Evaluation: Validate the model's effectiveness on a separate set of data that was not used during the training process. This helps to ensure that the model generalizes well to new, unseen data.
- Deployment: Once validated, the model is deployed as part of a plagiarism detection system where it can automatically analyze new submissions and flag potential instances of plagiarism.

Through supervised learning, plagiarism detection systems can achieve a high level of accuracy in identifying potential cases of code copying or paraphrasing, making it an invaluable tool in academic and professional settings[35]. This section sets the stage for a deeper exploration into the specific machine learning techniques we will employ, including deep learning models like CNNs and RNNs, which will be discussed in the following sections.

## 3.2 Deep Learning in Plagiarism Detection

Building on the foundational concepts of machine learning, this section delves into deep learning—a subset that uses complex neural networks to process data with multiple layers of abstraction. Deep learning models, particularly Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), have revolutionized the field of plagiarism detection by providing sophisticated tools to analyze the structural and sequential aspects of programming code [23], [26], [27], [33], [35], [39].

### 3.2.1 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks, commonly called CNNs, are specialized neural network architectures designed to process data with a grid-like topology[46]. This makes them particularly well-suited for dealing with spatial and temporal data, like images and videos, that maintain a high degree of correlation between adjacent elements [46].

CNNs are similar to other neural networks, but they have an added layer of complexity since they use a series of convolutional layers. Convolutional layers perform a mathematical operation called convolution, a specialized matrix multiplication, on the input data. The convolution operation helps to preserve the spatial relationship between pixels by learning image features using small squares of input data. The Fig 2 [46] represents a typical CNN architecture. The following are definitions of different

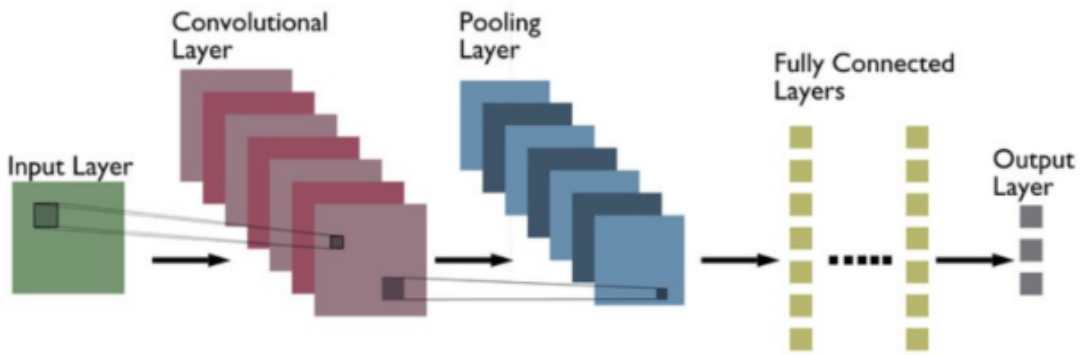


Fig. 2: CNN architecture

layers shown in the above architecture:

**Convolutional layers** operate by sliding a set of 'filters' or 'kernels' across the input data. Each filter is designed to detect a specific feature or pattern, such as edges, corners, or more complex shapes in the case of deeper layers [46]. As these filters move across the image, they generate a map that signifies the areas where those features were found. The output of the convolutional layer is a feature map, which represents the input image with the filters applied. Convolutional layers can be stacked to create more complex models, which can learn more intricate features from images. Simply speaking, convolutional layers are responsible for extracting features from the input images [46]. These features might include edges, corners, textures, or more complex patterns.

**Pooling layers** follow the convolutional layers and are used to reduce the spatial dimension of the input, making it easier to process and requiring less memory [46]. In the context of images, "spatial dimensions" refer to the width and height of the image. An image comprises pixels, and you can think of it like a grid, with rows

and columns of tiny squares (pixels). By reducing the spatial dimensions, pooling layers help reduce the number of parameters or weights in the network. This helps to combat overfitting and helps train the model quickly [35], [46]. Max pooling helps reduce computational complexity owing to the reduction in the size of the feature map and makes the model invariant to small transitions. Without max pooling, the network would not gain the ability to recognize features irrespective of small shifts or rotations. This would make the model less robust to variations in object positioning within the image, possibly affecting accuracy [46].

There are two main types of pooling: max pooling and average pooling. Max pooling takes the maximum value from each feature map [46]. For example, if the pooling window size is 2x2, it will pick the pixel with the highest value in that 2x2 region. Max pooling effectively captures the most prominent feature or characteristic within the pooling window. Average pooling calculates the average of all values within the pooling window. It provides a smooth, average feature representation [35], [46].

**Fully-connected layers** are one of the most basic layers in a convolutional neural network (CNN). As the name suggests, each neuron in a fully connected layer is Fully connected- to every other neuron in the previous layer [46]. Fully connected layers are typically used towards the end of a CNN- when the goal is to take the features learned by the convolutional and max pooling layers and use them to make predictions, such as classifying the input to a label. For example, if we were using a CNN to classify images of animals, the final Fully connected layer might take the features learned by the previous layers and use them to classify an image as containing a dog, cat, bird, etc [46].

Fully connected layers take the high-dimensional output from the previous convolutional and pooling layers and flatten it into a one-dimensional vector. This allows the network to combine and integrate all the extracted features across the entire image rather than considering localized features. It helps in understanding the global context of the image. The fully connected layers map the integrated features to the desired output, such as class labels in classification tasks. They act as the final decision-making part of the network, determining what the extracted features mean

in the context of the specific problem (e.g., recognizing a cat or a dog).

The combination of the Convolution layer followed by the max-pooling layer and then similar sets creates a hierarchy of features. The first layer detects simple patterns; subsequent layers build on those to detect more complex patterns [46].

**The output layer** in a Convolutional Neural Network (CNN) plays a critical role as it's the final layer that produces the network's actual output, typically in the form of a classification or regression result [35], [46]. Its importance can be outlined as follows:

- Transformation of Features to Final Output: The earlier layers of the CNN (convolutional, pooling, and fully connected layers) are responsible for extracting and transforming features from the input data. The output layer takes these high-level, abstracted features and transforms them into a final output form, directly interpretable in the context of the problem being solved [46].

- Task-Specific Formulation: The output layer typically uses a softmax activation function for classification tasks, which converts the input from the previous layers into a probability distribution over the predefined classes. The softmax function ensures that the output probabilities sum to 1, making them directly interpretable as class probabilities. For regression tasks, the output layer might consist of one or more neurons with linear or no activation function, providing continuous output values [46].

CNNs are often used for image recognition and classification tasks. For example, CNNs can be used to identify objects in an image or to classify an image as being a cat or a dog. CNNs can also be used for more complex tasks, such as generating descriptions of an image or identifying the points of interest in an image. Beyond image data, CNNs can also handle time-series data, such as audio data or even text data, although other types of networks, like Recurrent Neural Networks (RNNs) or transformers, are often preferred for these scenarios [33], [35], [46]. CNNs are a powerful tool for deep learning, and they have been used to achieve state-of-the-art results in many different applications.

For source code, CNNs can be trained to recognize textual and formatting patterns specific to programming languages, such as the use of particular keywords, coding style, indentation, and even the general layout that might suggest plagiarism.

### 3.2.2 Recurrent Neural Networks (RNNs)

RNNs are a type of neural network that can be used to model sequence data. RNNs, which are formed from feedforward networks, are similar to human brains in their behavior [47]. Simply said, recurrent neural networks can anticipate sequential data in a way that other algorithms can't. All of the inputs and outputs in standard neural networks are independent of one another; however, in some circumstances, such as when predicting the next word of a phrase, the prior words are necessary, and so the previous words must be remembered [47]. As a result, RNN was created, which used a Hidden Layer to overcome the problem. The most important component of RNN is the Hidden state, which remembers specific information about a sequence [46], [47]. RNNs have a Memory that stores all information about the calculations. It employs the same settings for each input since it produces the same outcome by performing the same task on all inputs or hidden layers [47].
RNN architecture can vary depending on the problem we are trying to solve. From those with a single input and output to those with many (with variations between). Below are some examples of RNN architectures [47]:

- One To One: There is only one pair here. A one-to-one architecture is used in traditional neural networks.
- One To Many: A single input in a one-to-many network might result in numerous outputs.
- Many To One: In this scenario, a single output is produced by combining many inputs from distinct time steps. Sentiment analysis and emotion identification use such networks, in which a sequence of words determines the class label.
- Many To Many: For many to many, there are numerous options. Two inputs

yield three outputs. Machine translation systems, such as English to French or vice versa translation systems, use many to many networks.

So, how do RNNs work? The information in recurrent neural networks cycles through a loop to the middle hidden layer. Let us look at Fig 3 [47] to understand better.

The input layer x receives and processes the neural network's input before passing

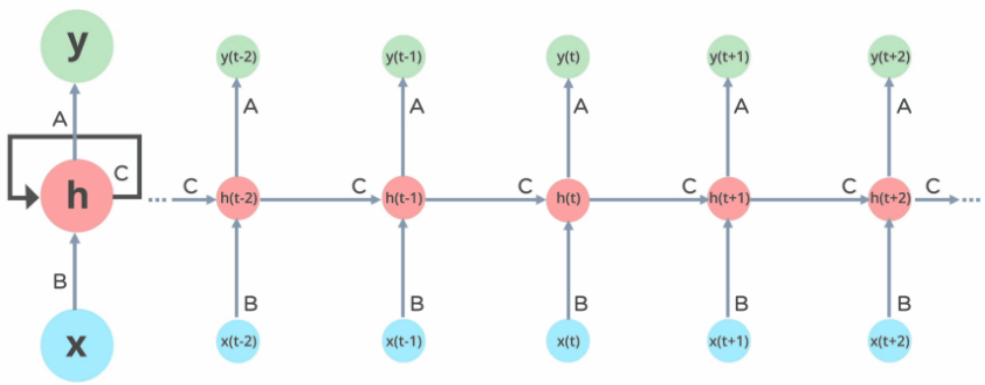


Fig. 3: RNN Architecture

it on to the middle layer. Multiple hidden layers can be found in the middle layer h, each with activation functions, weights, and biases. You can utilize a recurrent neural network if the preceding layer does not impact the various parameters of different hidden layers, i.e., there is no memory in the neural network [47]. The different activation functions, weights, and biases will be standardized by the Recurrent Neural Network, ensuring that each hidden layer has the same characteristics [47]. Rather than constructing numerous hidden layers, it will create only one and loop over it as many times as necessary.

By now, it is evident that Recurrent Neural Networks (RNNs) are a powerful class of neural networks designed to handle sequential data. Unlike traditional neural networks, RNNs have the unique feature of maintaining an internal state (memory) to process sequences of inputs [47]. This makes them ideal for tasks where the context and order of data are crucial, such as language modeling and text generation. However, standard RNNs often need help with long-term dependencies due to issues like

vanishing or exploding gradients during back-propagation [47]. This is where Long Short-Term Memory (LSTM) networks, a specialized type of RNN, come into play.

### 3.2.3 Long Short-Term Memory (LSTM) Networks

LSTM networks are an extension of RNNs that are specifically designed to overcome the limitations related to long-term dependencies [47], [49]. They achieve this through a complex architecture that includes several gates: input, output, and forget gates, which control the flow of information [47]. These gates determine whether to retain or discard information at each step of the sequence, allowing LSTMs to maintain a more stable gradient during training and thus learn from longer sequences [47]. The gating mechanism of LSTMs can decide what to store in long-term memory and what to discard, providing a way to clear out irrelevant past information. In the context of source code plagiarism detection, LSTMs can be particularly effective because of their ability to remember patterns over long sequences of code [49]. For example, they can track variable usage and manipulation across a program's execution, which might indicate code being rearranged to conceal plagiarism. The training process for LSTM involves adjustments in these gates and cell states, allowing the network to capture both short-term dependencies (such as local syntactic patterns in a block of code) and long-term dependencies (such as using a variable throughout an entire program). This dual capacity makes LSTMs exceptionally suited for analyzing the complexities of programming code within the scope of plagiarism detection.

The architectural nuances of LSTMs grant them the ability to discern subtle similarities in code structure and logic that might escape simpler models. This is essential in detecting sophisticated forms of plagiarism where the plagiarist may have consciously tried to disguise the copying through code modifications. Building on the capabilities of LSTM networks to understand sequential dependencies in code, let us now see how we can expand our analytical reach with Code Pre-trained Models (CodePTMs).

## 3.3 Code Pre-trained Models (CodePTMs)

Code Pre-trained Models (CodePTMs) have proven to be a beneficial innovation in source code analysis and have become instrumental in enhancing the semantic analysis capabilities of plagiarism detection systems [34]. These models are trained on large datasets of source code from various programming languages, allowing them to learn complex patterns and nuances specific to programming syntax and semantics [34], [37]. CodePTMs, such as CodeBERT and GraphCodeBERT, are adapted from general language models but are specifically fine-tuned on source code [37]. This training enables them to understand the structure of code and its intended functionality by analyzing comments, variable names, and function calls [37]. These models generate contextual embeddings that represent both syntactic and semantic information, which are crucial for identifying not just overt similarities but also subtler forms of semantic plagiarism. In the context of plagiarism detection, CodePTMs analyze code's textual and structural elements to detect patterns that may indicate plagiarism. They assess the semantic coherence of code snippets and compare these across various submissions to identify instances where the underlying idea or logic of the code is similar, even if the syntax differs. This capability is particularly valuable in detecting sophisticated forms of plagiarism where code may be significantly paraphrased or restructured. Integrating CodePTMs with CNNs and LSTMs leverages the strengths of each model, combining deep structural analysis with nuanced semantic understanding. By incorporating CodePTMs into our plagiarism detection framework, we hope to enhance our system's ability to interpret complex code structures and semantics, significantly boosting its effectiveness in identifying varied plagiarism tactics across different programming languages.

In our work, RNNs examine how different parts of the code interact over sequences, detecting outright copying and more subtle instances where the code might have been modified or restructured to conceal plagiarism. They are particularly effective in recognizing plagiarized code that has undergone modifications such as variable

renaming, line structure changes, or even logical flow alteration that simpler models might miss. Combining CNNs and RNNs leverages their strengths—structural pattern recognition and sequence processing. To this, we integrate CodePTMs, which can enhance our model's sensitivity to nuanced forms of plagiarism, such as those involving logical structures and algorithmic patterns that are not immediately obvious through conventional analysis methods. For example, CodePTMs can discern semantic similarities between code snippets that perform similar tasks but are implemented with different syntax or structure, thus bolstering our system's ability to detect sophisticated plagiarism tactics effectively. This hybrid approach is particularly potent for plagiarism detection in programming assignments because it allows the system to not only look at the static text of the code but also understand the dynamics of how the code is constructed over time. In summary, CNNs, RNNs, and NLP each play a crucial role in the advanced detection of plagiarism in source code [33], [35], [38], [40], [46], [47].

This background sets a foundation for the upcoming discussions on specific methodologies and experimental setups used in this research, bridging the gap between theoretical knowledge and practical application.

# CHAPTER 4

# *Proposed Methodology*

This chapter will explain the methodology we've developed to detect plagiarism in source code. We combine machine learning and natural language processing (NLP) techniques supported by detailed data analysis. We'll walk through each step, starting with gathering various source code files and ending with identifying and reporting instances of plagiarism.

Our model combines Recurrent Neural Networks (LSTM) for sequential data understanding with Convolutional Neural Networks (CNNs) for structural pattern recognition, bolstered by the NLP capabilities of a pre-trained CodePTM( specifically designed for source code analysis) to grasp contextual nuances[56]. By assessing code across syntax, structure, sequence, and semantics, our system achieves a nuanced analysis that detects plagiarism tactics. We also understand the significance of context when assessing plagiarism. Therefore, our approach involves analyzing metadata associated with code submissions. This includes details like timestamps, file size, any changes in file size between submissions, and other relevant data, providing valuable insights to identify potential instances of plagiarism. These features are extracted from each submission and are embedded into the feature vector, which is used in step 3(in Fig 4) while training the model.

Before getting into the details of each step in our proposed methodology, let's look at Fig 4, which provides an overview of the stages involved in our proposed solution. Our methodology is divided into four steps: data preprocessing(Step 1), feature ex-

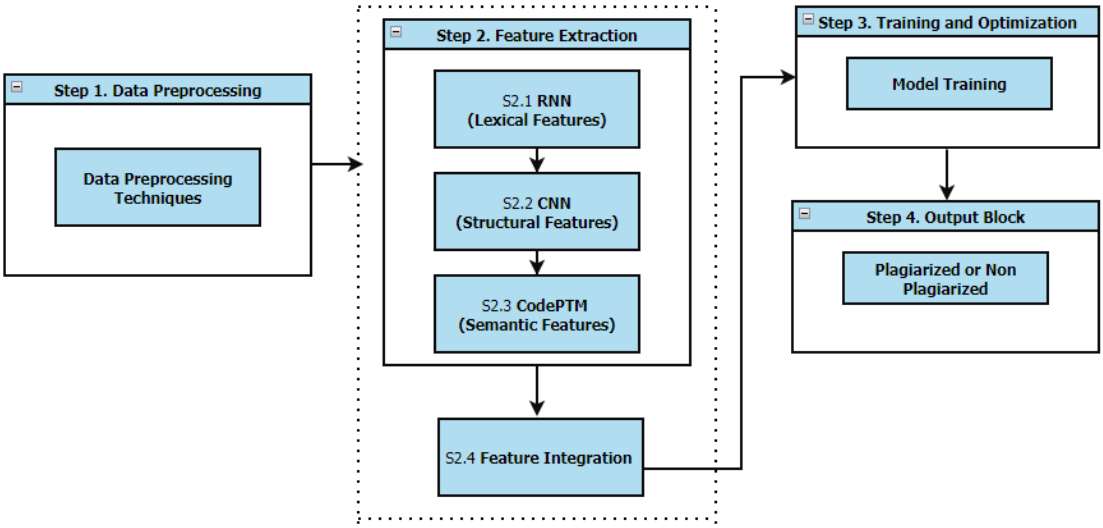


Fig. 4: Proposed Methodology

traction(Step 2), training and optimization(Step 3), and the output block(Step 4). Our methodology's first step(step 1 in Figure 4) focuses on data preprocessing. This step involves cleaning our dataset and preparing the source code for analysis by applying preprocessing techniques, tokenization, and AST generation.

The second step(Step 2 in Fig 4) in our proposed methodology is feature extraction, which is divided into four stages: RNN(S2.1 in Fig 4), CNN(S2.2 in Fig 4), CodePTM(S2.3 in Fig 4), and feature integration(S2.4 in Fig 4). Here, RNN(LSTM) extracts lexical features, analyzing the sequence and syntax of the code. Convolutional Neural Networks identify structural patterns and similarities in the source code. Pre-trained NLP model CodePTM extracts semantic features that provide insights into the contextual meanings behind code segments. Finally, the feature integration stage(S2.4 in Fig 4) involves combining the lexical, structural, and semantic features into a unified feature vector that will be fed into the machine-learning model for classification. This feature vector consolidates all the crucial attributes extracted from the source code, making it a comprehensive dataset that the machine learning model can analyze to determine if a piece of code is plagiarized. This vector is crucial as it represents a fusion of all significant data points necessary for plagiarism detection.

The third step (Step 3 in Fig 4) of our methodology focuses on training and optimizing the machine learning model to detect plagiarism in source code. This step involves training a logistic regression model, a simple yet effective binary classifier. The training process utilizes a single, consolidated feature vector that results from integrating all extracted features at the end of Step 2. This comprehensive feature vector is fed into the logistic regression algorithm to learn the distinctions between plagiarized and non-plagiarized code segments. We optimize the model through the adjustment of hyperparameters. The model utilizes the Adam optimizer as part of the logistic regression's configuration.

With that, we move on to our final step(step 4 in Fig 4), the output block. This is the decision-making stage where our trained model from step 3 classifies the pairs of source code as either plagiarized or not, based on the learned patterns and similarities in their features.

Now that we have an overall idea of the workflow of our proposed solution, we would like to look at each of these steps and stages in greater detail in subsequent sections.

## 4.1 Data Preprocessing

Before we talk about the data preprocessing step, it is essential to understand the dataset. The dataset, fundamental to our plagiarism detection approach, was procured from IEEE DataPort. This dataset comprises student homework submissions from introductory programming courses conducted between 2016 and 2017[41]. It includes 54K C program files, of which 15,778 are marked plagiarized[41]. To clarify, 15,778 refers to individual files marked as plagiarized, not pairs of files. Each of these files represents a separate instance of plagiarism. The dataset is divided into two main sections:

- Source Code: This section contains the actual programming assignments students submit. The dataset we used showcases a wide range in the size of submissions, with the smallest program consisting of merely 10 lines of code, while the largest is 2,024 lines. On average, a program file in our dataset contains around 150 lines of code, reflecting the diverse complexity and scope of the assignments[52].

- Ground Truth: The Ground Truth data serves as a crucial benchmark for the effectiveness of our plagiarism detection approach. It comprises 15,778 instances of student submissions, which were flagged for potential plagiarism based on a range of similarities. These similarities span from identical or nearly identical code snippets to subtle, less obvious indicators such as similar problem-solving approaches or logic flow despite differences in code syntax[52].

On this dataset [52], we applied random masking to introduce noise, which helps avoid overfitting while training. For this, we programmatically altered 5% of our dataset by masking random code segments. Our dataset underwent a random alteration process, where code segments were programmatically obscured. In applying the random masking technique to our dataset, we carefully considered the potential implications of making code segments appear superficially similar, which could raise suspicions of low-level plagiarism. However, the random masking was applied systematically and non-discriminately across the selected 5% of the dataset, ensuring that any masked segment does not replicate specific patterns characteristic of plagiarism, such as consistent variable renaming or structural mimicry found in deliberately plagiarized code. Instead, the masking was designed to obscure random portions of code, such as a variable declaration or a loop structure, without creating coherent or meaningful patterns that would typically be associated with intentional plagiarism. A sample C program and a randomly masked variant can be seen in Fig This method aims to introduce noise and variation into the dataset to test the robustness of our plagiarism detection algorithm against non-plagiaristic alterations, enhancing its ability to distinguish between genuine instances of plagiarism and mere similarities due to common pro-

```c
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;
    printf("The sum is: %d\n", sum);
    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int a = 10;
    int b = MASKED;
    int sum = a + b;
    printf("The sum is: %d\n", sum);
    return 0;
}
```

Fig. 5: Sample C program and randomly masked version

gramming constructs or coincidental alignments. After applying random masking, we retained a clear record of the origin of each file, ensuring that the count of files classified as plagiarized before masking remained accurately documented post-masking. This approach allows us to preserve the exact count and categorization of plagiarized files, regardless of the masking process. As a result, even after expanding our dataset, the number of files originally marked as plagiarized remains precisely accounted for, providing a reliable base for further analysis and validation of our plagiarism detection methods. So, with our dataset containing 54,000 C programs, out of which 15,778 submissions are marked plagiarized, with the levels of plagiarism ranging from Level 1 to Level 6(shown as a pie chart in Fig 5), we then proceed for further processing steps. This pie chart illustrates the distribution of 15,778 plagiarized C programs categorized across six levels of plagiarism complexity. The classification is based on the nature and subtlety of the plagiarism involved:

- Level 1: Textual Replication (20.3%, 3,200 files) includes cases where the code is copied without any changes, making it the most straightforward to detect.
- Level 2: Syntactic Alteration (19.0%, 3,000 files) involves superficial changes such as renaming identifiers or altering comments while maintaining the underlying code structure.
- Level 3: Structural Modification (17.7%, 2,800 files) covers subtle yet significant changes to the code structure, such as modifications to loops and conditionals that preserve the original logic.

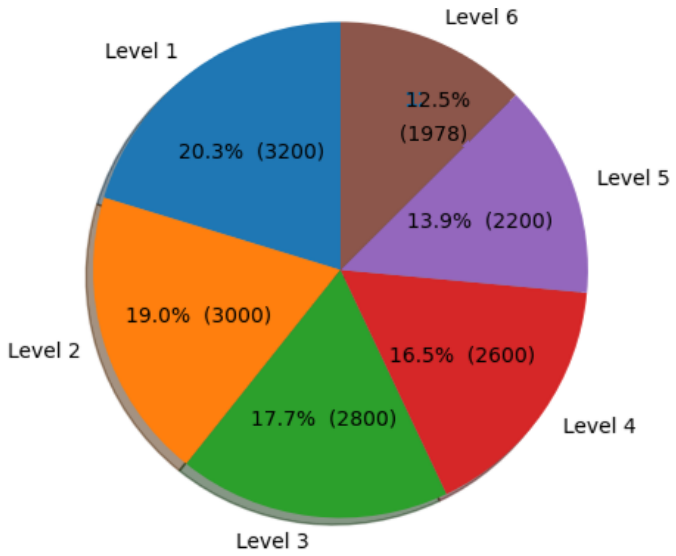


Fig. 6: Distribution of Plagiarism Complexity Levels in our Dataset

- Level 4: Control Flow Transformation (16.5%, 2,600 files) represents more complex alterations involving the sequence or organization of code blocks and often requires control flow analysis to detect.
- Level 5: Algorithmic Paraphrasing (13.9%, 2,200 files) refers to cases where the underlying algorithms are rewritten to achieve similar functionality in a non-obvious manner.
- Level 6: Design-Level Mimicry (12.5%, 1,978 files) is the most sophisticated and difficult level, involving copying or mimicking just the overall design or problem-solving approach, demanding an in-depth understanding of the program's domain and intent.

This distribution helps highlight the varying degrees of sophistication in plagiarism practices. It allowed our model to encounter and learn from a breadth of coding expressions, increasing its adeptness at pinpointing overt and sophisticated forms of code plagiarism. It's crucial to note that the variations created through random masking are used solely for training purposes. They provide a way for our model to differentiate between simple stylistic changes and actual plagiaristic alterations, ensuring robust detection accuracy in practical scenarios [56]. This comprehensive

dataset provided the foundation for developing a plagiarism detection system that is well-versed in the intricacies of programming assignments and capable of identifying plagiarism with high accuracy[36].

With the dataset ready, we move on to step 1 of our proposed methodology, the data preprocessing technique(Step 1 in Fig 4). Effective data preprocessing forms the foundation of our plagiarism detection system, ensuring that raw code submissions are transformed into a format conducive to detailed analysis by our machine learning models. This step (Step 1 in Fig 4) begins by cleaning and normalizing the code submissions to ensure consistency across the dataset and focus solely on the functional aspects of the code. This includes the removal of Personally Identifiable Information (PII) using automated scripts to strip out any personal data associated with the students, such as names, student ID numbers, email addresses, etc. This not only protects student privacy but also eliminates irrelevant data that could distort the analysis [32], [35], [56]. Following the initial cleanup and normalization of the source code, the next important phase in our data preprocessing workflow involves tokenization and the structuring of these tokens for comprehensive analysis. We employ the Clang tokenizer, a core component of the Low-Level Virtual Machine (LLVM) project, renowned for its modular and reusable compiler and toolchain technologies [22], [16]. In the process of converting source code tokens into vector representations, also known as token embeddings, it is essential to capture not only the syntactic essence but also the broader semantic context of each token. To achieve this, the Clang tokenizer first breaks down the source code into a series of tokens(also seen in Fig 6), which include keywords, operators, and literals. Each token is then transformed into a vector representation using an embedding technique. For our purposes, we employ the Word2Vec model, a widely used approach in natural language processing and programming language analysis. Word2Vec maps each unique token to a high-dimensional space where semantically similar tokens are positioned closer to each other. This model is trained on a large corpus of source code, allowing it to learn rich, contextual embeddings for each token based on its usage and co-occurrence

```c
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;
    printf("The sum is: %d\n", sum);
    return 0;
}
```

```
('#', 'PUNCTUATION')
('include', 'IDENTIFIER')
('<', 'PUNCTUATION')
('stdio', 'IDENTIFIER')
('.', 'PUNCTUATION')
('h', 'IDENTIFIER')
('>', 'PUNCTUATION')
('int', 'KEYWORD')
('main', 'IDENTIFIER')
('(', 'PUNCTUATION')
(')', 'PUNCTUATION')
('{', 'PUNCTUATION')
('int', 'KEYWORD')
('a', 'IDENTIFIER')
('=', 'PUNCTUATION')
('10', 'LITERAL')
(';', 'PUNCTUATION')
('int', 'KEYWORD')
('b', 'IDENTIFIER')
('=', 'PUNCTUATION')
('20', 'LITERAL')
(';', 'PUNCTUATION')
('int', 'KEYWORD')
('sum', 'IDENTIFIER')
('=', 'PUNCTUATION')
('a', 'IDENTIFIER')
('+', 'PUNCTUATION')
('b', 'IDENTIFIER')
(';', 'PUNCTUATION')
('printf', 'IDENTIFIER')
('(', 'PUNCTUATION')
('"The sum is: %d\\n"', 'LITERAL')
(',', 'PUNCTUATION')
('sum', 'IDENTIFIER')
(')', 'PUNCTUATION')
(';', 'PUNCTUATION')
('return', 'KEYWORD')
('0', 'LITERAL')
(';', 'PUNCTUATION')
('}', 'PUNCTUATION')
```

Fig. 7: Sample C program and generated tokens

with other tokens. These embeddings are designed to capture not only the syntactic essence of each token but also the broader semantic context, which is critical for understanding the underlying meaning and functionality of the code. This process produces a rich dataset of embeddings(an example can be seen in Fig 7 ) that serve as a foundational input for LSTM (S2.1 in Fig 4) and CodePTM (S2.3 in Fig 4). The LSTM model utilizes these embeddings to capture temporal and contextual dependencies within code sequences. This sequential processing allows the LSTM to discern patterns and anomalies indicative of potential plagiarism, leveraging the order and context in which tokens appear. Similarly, these embeddings are fed into a Code Pre-trained Model (CodePTM), which is fine-tuned to grasp both general pro-

```
Token Embeddings:
Token: #, Embedding: [-0.04509399  0.05701705  0.09180015 -0.04099872  0.07964681  0.05375434
  0.05879123  0.0051259   0.08213083 -0.07019041]
Token: include, Embedding: [-0.00853795  0.02822798  0.05397586  0.07050323 -0.05704837  0.0185667
  0.06087504 -0.0480092  -0.03107982  0.06797647]
Token: <, Embedding: [ 0.01627435  0.00189649  0.03469095  0.0021749   0.09624835  0.05059974
 -0.08912453 -0.0704535   0.00897493  0.06397954]
Token: stdio.h, Embedding: [-0.08625071  0.0365785   0.05189146  0.05742403  0.07474167 -0.06176585
  0.01118853  0.06048171 -0.02850637 -0.06170587]
Token: >, Embedding: [-0.0040505  -0.08380144 -0.05604869  0.07103441  0.03348338  0.07223572
  0.06807385  0.07530084 -0.03788326 -0.0055942 ]
Token: int, Embedding: [ 0.07380505 -0.01533471 -0.04536613  0.06554051 -0.0486016  -0.01816018
  0.0287658   0.00991874 -0.08285215 -0.09448818]
Token: main, Embedding: [ 0.02343238 -0.04527698  0.08386982 -0.09857606  0.06769906  0.02904582
 -0.04919428  0.04399281 -0.01749182  0.06714223]
Token: (, Embedding: [-0.08166078  0.04497485 -0.04143766  0.00825022  0.08508059 -0.04464595
  0.04525103 -0.06793713 -0.03553249  0.09408403]
Token: ), Embedding: [-0.01577653  0.00321372 -0.0414063  -0.07682689 -0.01508008  0.02469795
 -0.00888027  0.05533662 -0.02742977  0.02260065]
```

Fig. 8: Example of token embeddings

gramming language patterns and specific semantic nuances of the source code. Unlike the LSTM, CodePTM employs a broader context analysis, not limited to sequential dependencies but encompassing wider contextual relationships. This capability enables CodePTM to identify deep semantic similarities and discrepancies that might not be apparent from structural analysis alone, thus enhancing our model's ability to detect sophisticated forms of semantic plagiarism. By employing the Clang tokenizer and transforming the output into high-quality token embeddings, we ensure that the LSTM and CodePTM have the most informative and contextually enriched data possible. Next, we generate Abstract Syntax Trees (ASTs) for each program by utilizing Clang's advanced parsing capabilities, specifically tailored for the C language. These ASTs provide a hierarchical representation (also seen in Fig 8) of the code structure, which is fundamental for analyzing the logical and structural relationships within the code. To prepare these ASTs for convolutional neural network (CNN) analysis, we undertake a systematic transformation. Each node of the AST is encoded based on its type (e.g., operator, identifier, conditional) and attributes (e.g., depth in the tree, number of children). This encoding converts complex tree structures into a more manageable form, preserving essential syntactic and structural information. We then organize these encoded nodes into a matrix format(example shown in Fig 9). This matrix setup allows us to maintain the spatial hierarchy of nodes, mirroring their

```c
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;
    printf("The sum is: %d\n", sum);
    return 0;
}
```

```
TRANSLATION_UNIT: sample.c
  FUNCTION_DECL: main
    COMPOUND_STMT:
      DECL_STMT:
        VAR_DECL: a
          INTEGER_LITERAL:
      DECL_STMT:
        VAR_DECL: b
          INTEGER_LITERAL:
      DECL_STMT:
        VAR_DECL: sum
          BINARY_OPERATOR:
            UNEXPOSED_EXPR: a
              DECL_REF_EXPR: a
            UNEXPOSED_EXPR: b
              DECL_REF_EXPR: b
      CALL_EXPR: printf
        UNEXPOSED_EXPR: printf
          DECL_REF_EXPR: printf
        UNEXPOSED_EXPR:
          UNEXPOSED_EXPR:
            STRING_LITERAL: "The sum is: %d\n"
        UNEXPOSED_EXPR: sum
          DECL_REF_EXPR: sum
      RETURN_STMT:
        INTEGER_LITERAL:
```

Fig. 9: Sample C program and generated AST

arrangement in the code. Within this matrix, each node's features, such as type, depth, and connectivity, are represented in distinct channels or sections of the matrix, similar to how image data is represented in different color channels in CNNs for image processing. This multi-dimensional representation helps in capturing both local and global structural patterns effectively. This processed AST matrix serves as the input for our CNN (discussed in section 4.2), where they are utilized to detect patterns in the code structure that may indicate plagiarism. By converting the ASTs into a CNN-compatible format, we leverage the network's ability to perform complex pattern recognition, enhancing our plagiarism detection capabilities.

These preprocessing efforts (step 1 in Fig 4) are vital because cleaning and normalizing the code ensures that the machine-learning models focus on meaningful code structures and patterns. While tokenization aids in identifying syntactic similarities, ASTs help recognize structural patterns often manipulated in plagiarism attempts. At the close of Section 4.1, our data preprocessing efforts yield token embeddings (for LSTM) and AST matrix (for CNN). These pave the way to section 4.2, where step 2 (in Fig 4), feature extraction begins.

```
AST Matrix:
[[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0]
 [0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 5 0]
 [0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 3 1 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 4 1 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 5 0 0]
 [0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 3 1 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 4 1 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 5 0 0]
 [0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 3 1 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 4 1 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 5 2 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 6 1]
 [0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 7 0]
 [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 6 1]
 [0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 7 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 3 3]
 [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 4 1]
 [0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 5 0]
 [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 4 1]
 [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 5 1]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 6 0]
 [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 4 1]
 [0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 5 0]
 [0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 3 1]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 4 0]]
```

Fig. 10: AST in a matrix representation

## 4.2 Feature Extraction

This section will discuss step 2(in Fig 4) of our proposed methodology, which is divided into four stages (S2.1, S2.2, S2.3, S2.4 in Fig 4). After preprocessing the source code in step 1(in Fig 4), feature extraction stages 1-3 (S2.1, S2.2, S2.3), where we use LSTM, CNN, and NLP would help us extract the features, and stage 4 (S2.4 in Fig 4), which is feature integration will help us to combine the lexical, structural, and semantic features into a unified feature vector that will be fed into the machine-learning model for classification. Let us look into the details of these stages.

In the lexical analysis phase (S2.1 in Fig 4), we use a bi-directional LSTM (Bi-LSTM) layer to analyze sequences of token embeddings obtained from step 1(Fig 4). Each token's embedding captures its syntactic nature, allowing the Bi-LSTM to process the sequence forward and backward. This dual-direction capability is essential for capturing the full context of the tokens and identifying both preceding and succeeding syntactic elements, which is critical for detecting plagiarism. The Bi-LSTM layer has 128 hidden units, allowing it to maintain a detailed context of previously encountered tokens throughout the code snippet. As the Bi-LSTM processes the token

embeddings, its cells dynamically update their internal states based on the current input token and the contextual information from adjacent tokens. This results in the output of a sequence of hidden states that reflect the temporal dependencies and overall context of the tokens within the snippet. This layer is particularly effective in recognizing patterns of code that may have been rearranged or subtly modified, a common tactic in sophisticated plagiarism attempts. By analyzing the order and context in which tokens appear, the Bi-LSTM can detect not just direct copying but also more insidious forms of plagiarism, such as those involving the reordering of function calls or operational sequences to mask the copying of logic or functionality. Consequently, the system is adept at identifying similarities in code structure and logic across diverse code snippets, even when superficial aspects of the code appear different. With this analytical approach, lexical features ($L$) are derived from the sequence of token embeddings, represented as $L : P \rightarrow \mathbb{R}^n$, where $P$ is token embeddings from step 1 and $\mathbb{R}^n$ denotes an n-dimensional vector space of lexical features. These features capture and quantify the lexical patterns critical for distinguishing between plagiarized and non-plagiarized code segments.

After this, we move on to perform structural analysis (S2.2 in Fig 4) using CNN. The Convolutional Neural Network (CNN) in our model processes the structured matrix representation derived from the ASTs (S2.2 in Fig 4). The CNN is structured with three convolutional layers, each followed by a max-pooling layer, designed to extract structural features from the code's syntax. The first convolutional layer uses 32 filters of size 3x3, targeting basic structural elements such as whitespace, braces, and brackets. These features are common in source code formatting and provide foundational spatial cues for the CNN. As data passes through subsequent convolutional layers, the network delves deeper into the code's architecture, aiming to identify more complex structural motifs like function blocks and control structures. The AST matrices serve as the input for the CNN. During the convolutional process, these layers traverse the matrix, generating feature maps that highlight areas of structural similarity and anomaly. This step is critical in identifying recurring coding

patterns and discrepancies that might suggest plagiarism. The network's ability to analyze the spatial arrangements within the matrix allows it to understand subtle structural relationships and organizational patterns in the code. From these feature maps, structural features ($S$) are extracted and quantified into an m-dimensional vector space $\mathbb{R}^m$ where $\mathbb{R}^m$ is a space where each point (or vector) is described by $m$ real numbers. Each dimension in this space can represent a quantifiable feature extracted from the AST, such as the depth of the tree, the types of nodes, the frequency of certain patterns, or other statistical or structural properties derived from the AST. These features $S : A \rightarrow \mathbb{R}^m$, where $A$ encapsulates the structural representation of the source code as an AST. ASTs are tree-based data structures that represent the hierarchical syntax of programming languages. Each node in an AST corresponds to a construct occurring in the source code, such as declarations, statements, expressions, etc., providing a robust basis for identifying potential plagiarism by comparing these patterns across various code submissions.

In the subsequent phase (S2.3 of Fig 4), we utilize Code Pre-CodePTM to extract semantic features from the source code. CodePTM, fine-tuned on a vast corpus of programming data, excels at deciphering general language patterns and specific programming semantics. This fine-tuning equips CodePTM with the capability to understand and interpret various elements of source code deeply. So, to perform this analysis, we feed our token embeddings (from Step 1) to our CodePTM. CodePTM processes these token embeddings to analyze crucial semantic elements within the code. It focuses on parts that carry significant semantic weight, such as variable names and comments, transforming these textual elements into contextual embeddings. These embeddings encapsulate the deeper semantic meanings and the relationships between different parts of the code. By analyzing these contextual embeddings, CodePTM significantly enhances our system's ability to detect semantic plagiarism. The high-dimensional vector representations generated by CodePTM provide insights into the code's purpose and functionality. For instance, these embeddings might reveal semantic similarities between code segments that perform similar operations but appear syntactically different. This capability is invaluable for detecting plagiarism

that modifies structural elements but retains the original code's intent and functionality. CodePTM converts these token embeddings into a k-dimensional vector space of semantic features ($Se$) formalized as $Se : P \rightarrow \mathbb{R}^k$ with ($P$) being preprocessed tokens and the $\mathbb{R}^k$ representing a k-dimensional vector space for semantic features [56].

Before moving on to stage 4 (S2.4 in Fig), let us summarize the feature extraction stages (S2.1, S2.2, and S2.3). The input for the CNN consists of preprocessed source code data represented as matrices derived from Abstract Syntax Trees (AST). These matrices are structured to maintain the hierarchical and spatial relationships inherent in the source code, facilitating the extraction of structural patterns. The CNN processes this matrix input through several convolutional and pooling layers, extracting local and broader structural patterns from the code. The output from the CNN is a set of feature maps that highlight relevant features at various levels of abstraction, encoding critical information about the code's syntax and structure, which are essential for identifying structural similarities and anomalies. Moving on, the LSTM receives a sequential representation of token embeddings from the source code. Each token is transformed into a numerical vector representation, forming a sequence that captures the temporal and contextual dependencies of the code. The LSTM processes this sequential input through its recurrent architecture, learning to identify and capture long-range dependencies and contextual nuances within the code. The output of the LSTM is a sequence of hidden states, each representing an aggregated understanding of the input tokens up to that point, encoding the learned representations crucial for detecting patterns indicative of plagiarism. Then, CodePTMs receive a sequence of token embeddings. Leveraging advanced self-attention mechanisms, CodePTM analyzes these embeddings to capture both local and global dependencies and contexts within the code. This model is adept at understanding deeper semantic relationships due to its ability to consider the entire code context simultaneously. The output from CodePTM consists of high-level representations that encode comprehensive semantic and syntactic information of the code. These representations, rich in contextual understanding, are especially valuable for tasks like plagiarism detection, where un-

derstanding the intent and functionality beyond mere syntax is crucial.

After extracting the diverse set of features, we integrate them into a unified representation (S2.4 in Fig 4), suitable for input into our machine learning models. This integrated feature vector combines lexical, structural, and semantic features to create a comprehensive representation that encapsulates multiple dimensions of the source code. The integration is computed as $F = L(P) \oplus S(A) \oplus Se(P)$, where $\oplus$ represents the concatenation operation. To perform this concatenation, each output is first transformed into a fixed-length representation to concatenate the outputs of different feature extraction methods (CNN, LSTM, CodePTM) into a single feature vector. The output feature maps from the CNN, which capture the structural patterns within the code, are flattened into a vector representation. This process converts the multi-dimensional feature maps into a single-dimensional vector that retains essential structural information. The LSTM produces a sequence of hidden states that sequentially represent the input tokens. To derive a fixed-length representation suitable for integration, we apply a pooling operation over these states. This results in a single vector that effectively summarizes the entire sequence's contextual dependencies. Similarly, the output from CodePTM provides a high-level representation encapsulating the semantic and syntactic nuances of the code. We also apply a pooling operation to these outputs to achieve a fixed-length semantic feature vector. Once we have obtained fixed-length representations from each feature extraction method, we concatenate these vectors to form a unified feature vector($F$).

So, until now, we have extracted lexical, structural, and semantic features from the source code and generated high-dimensional vectors for each feature type. Each feature type contributes a distinct dimensionality to the overall feature vector:

- Lexical features based on the tokenizer's vocabulary include around 300 dimensions, covering various keywords, operators, and syntax types used in C programming.

- Structural features derived from the Abstract Syntax Trees (ASTs) contribute

about 150 dimensions, representing different node types and their hierarchical relationships.

- Semantic features, encoded by CodePTMs, capture nuanced programming semantics with around 200 dimensions, emphasizing contextual dependencies and subtle coding intents.

When these feature types are combined, the concatenated feature vector ($F$) results in 650 dimensions (300 for lexical, 150 for structural, and 200 for semantic). Managing such high dimensionality can lead to issues like overfitting during model training. To mitigate and address this, we apply Principal Component Analysis (PCA) to reduce the feature space to a more manageable size, 356 dimensions. This reduction helps ensure that our binary classification model(logistic regression) remains computationally efficient and focuses on the most significant features for detecting plagiarism, thereby enhancing the model's accuracy and efficiency. So, for a given pair of C programs, our approach does the following :

- Data Preprocessing(Step 1 in Fig 4): We clean and standardize the source code. This involves tokenizing the code into manageable tokens (token embeddings, example seen in Fig 7) and creating Abstract Syntax Trees (seen in Fig 8), which are converted to the matrix (seen in Fig 9) to understand its structure.

- Feature Extraction(Step 2 in Fig 4): Next, we load AST as input for CNNs to identify structural patterns and token embeddings to LSTM networks to capture temporal patterns of the code sequences and token embeddings to CodePTMs to understand the semantic meaning behind the code. We then combine the features from the previous steps into a single feature vector. This vector includes lexical (from tokens), structural (from ASTs), and semantic (from CodePTM) attributes.

Having detailed the extraction and integration of our feature vector($F$) that encapsulates the source code's lexical, structural, semantic, and metadata features, we now focus on the model that employs this vector for plagiarism detection. The following

section will introduce the binary classification model chosen for this task, explaining why this specific model fits our requirements and how it utilizes the feature vector for decision-making.

We will deploy a logistic regression classifier, favored for its efficiency in binary classification tasks, to interpret our integrated feature vector. This model was selected because of its proven track record in similar applications where a clear decision boundary between two classes (plagiarized vs. non-plagiarized) is essential[23], [55]. The logistic regression model will process the feature vector ($F$) to calculate the probability of plagiarism and classify each pair of source code submissions accordingly. Following this, the discussion will transition to the training and optimization phase, where we will fine-tune our model to enhance its learning efficacy and ensure it generalizes well to new, unseen code samples, thus maintaining its effectiveness across diverse programming scenarios.

## 4.3 Training and Optimization

Moving on to step 3 in Fig 4, which is training and optimization. In our plagiarism detection system, the binary classification model employed is a Logistic Regression classifier. This choice is due to its effectiveness in handling binary outcomes, which, in our case, distinguishes between "plagiarized" and "non-plagiarized" pairs of source code files. Logistic Regression is particularly suited for this task because it provides a probabilistic framework, allowing the model to calculate the likelihood of plagiarism based on the input feature vector derived from CNN, LSTM, and CodePTM outputs in step 2. This setup allows us to generate a score that quantifies the probability of a code pair being plagiarized, facilitating threshold-based decision-making in the final evaluation stage(step 4).

In our methodology, the dataset comprising 54,000 C programs was allocated into training, validation, and testing sets with proportions of 70%, 15%, and 15%, respec-

| | Total Files | Non-Plagiarized | Plagiarized | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | L1 | L2 | L3 | L4 | L5 | L6 |
| Training Set | 37,800 | 26,756 | 2240 | 2100 | 1960 | 1820 | 1540 | 1384 |
| Validation Set | 8,100 | 5,733 | 480 | 450 | 420 | 390 | 330 | 297 |
| Test Set | 8,100 | 5,733 | 480 | 450 | 420 | 390 | 330 | 297 |

Fig. 11: Dataset Split

tively. These data split distributions can also be seen in Fig 10. The distribution results in 37,800 files for training, 8100 files for validation, and 8100 files for testing. From Fig 10, we can also see how the plagiarized files are further divided based on the levels of plagiarism in each file. The training utilized mini-batches of 128 samples to optimize computational resources while ensuring stable gradient updates, balancing GPU memory use, and maintaining batch representativeness. This approach accelerates convergence and enhances the model's ability to generalize effectively across diverse plagiarism instances, leveraging the full spectrum of the feature vector ($F$) to detect plagiarism in C programs more reliably[53], [56]. Our machine learning process begins by selecting a suitable model and corresponding loss function. We chose a Logistic Regression classifier for our plagiarism detection system due to its efficiency in binary classification tasks. This model uses binary cross-entropy as the loss function, which was chosen to measure the probability divergence between actual class labels and predictions. This setup quantifies prediction errors effectively, enhancing the model's ability to discern between 'plagiarized' and 'non-plagiarized' instances.

We establish the training setup once the model and its loss function are defined. The training involves adjusting the model parameters to minimize the binary cross-entropy loss, thereby refining the model's ability to identify subtle nuances of code plagiarism accurately. This sets the foundation for subsequent training iterations where the logistic model's parameters are optimized, guided by the loss function. This structured approach ensures that each component of the training process contributes effectively to the model's overall discriminatory power, enabling it to detect

a wide range of plagiarism instances reliably. We also move into optimizing the training of our logistic regression model. This step encompasses hyperparameter tuning, where we adjust settings such as learning rate and the number of epochs to fine-tune the model's performance, preventing overfitting and ensuring robust generalization. We set the learning rate to 0.001 for steady convergence without overshooting the minimum loss. The model is trained for up to 50 epochs, with an early stopping mechanism triggered if no improvement in validation loss is observed, optimizing for performance. Additionally, we employed a dropout rate of 0.5 in the LSTM and CNN layers to prevent overfitting by randomly omitting units during training, enhancing the model's ability to generalize. Through tuning and testing, we established an optimal configuration that maximized the accuracy and minimized the error on the validation set, preparing the model for effective real-world application. The machine learning model is optimized using the Adam optimizer, known for its robustness and efficiency in handling sparse gradients on noisy problems. The Adam optimizer is particularly suited for our dataset due to its adaptive learning rate capabilities, governed by the hyperparameters $\beta_1$ and $\beta_2$. $\beta_1$, typically set at 0.9, controls the decay rate of the moving averages of past gradients, smoothing out the updates and ensuring consistency across iterations. $\beta_2$, often set at 0.999, manages the decay rate of moving averages of past squared gradients, stabilizing the convergence by adapting the learning rate based on the geometry of the data[56].

As we conclude the optimization of our machine learning model, we prepare to demonstrate its application in the output block (Step 4 from Fig 4), where the model's effectiveness in real-world scenarios will be showcased.

## 4.4 Output and Decision Making

In step 4(in Fig 4), we will focus on how our model utilizes the processed data to conclude the plagiarism status of source code submissions. After training and optimizing our model, it's critical to understand how it synthesizes complex data into action-

able results. Our plagiarism detection model operates as a binary classifier where the feature vector ($F$) is input to determine if a pair of code samples is plagiarized. The feature vector ($F$) that represents different dimensions of the code is analyzed, offering a holistic view that enhances the detection capabilities. The feature vector ($F$) is processed by a logistic regression model, which evaluates the probability of plagiarism. The logistic function outputs a score $Y$, which is compared against a predefined threshold to determine the final classification:

$$\text{Output} = \begin{cases} \text{Plagiarized}, & \text{if } Y \geq \text{threshold} \\ \text{Non-Plagiarized}, & \text{otherwise} \end{cases}$$

The threshold for identifying plagiarism within our model is carefully calibrated using the training data to optimize recall and precision. This calibration ensures that our model effectively identifies cases of plagiarism without excessively penalizing legitimate code. We have systematically experimented with different threshold values to strike an optimal balance. These experiments were conducted using a dedicated validation set, which allows us to observe the impact of each threshold adjustment on model performance, thus guiding us in selecting the most effective setting for our needs. To further refine our model's output, we calculate a plagiarism percentage to measure the extent of similarity, providing a quantitative basis for the plagiarism claim, which supports the binary decision. This approach ensures that each step—from feature integration to final decision-making—is grounded in robust statistical methods and machine learning practices, enhancing the reliability and validity of the plagiarism detection system.

The subsequent chapter will delve into the empirical validations of these mechanisms, comparing our model's performance against established benchmarks in the field.

# CHAPTER 5

# *Experiments and Results*

This chapter discusses validating our proposed methodology for source code plagiarism detection. We assess the system's efficacy in identifying plagiarized code across our dataset through a series of experiments. The primary objective of these experiments is to evaluate the effectiveness of combining feature extraction methods(CNNs, LSTM, and CodePTM) in detecting various forms of plagiarism in source code, specifically within C programming assignments. The experiments aim to validate the hypothesis that the feature extraction technique, which integrates lexical, structural, and semantic analysis, can provide better results than single-method approaches in detecting plagiarism. The experimental outcomes substantiate previous chapters' theoretical underpinnings and methodological approaches.

## 5.1 Experimental Setup

The experiments were conducted using the settings listed below to ensure the efficiency and scalability of the plagiarism detection system. The key hardware and software specifications include:

- CPU: Intel Xeon Processor E5-2686 v4 (2.3 GHz, 16 cores)
- GPU: NVIDIA Tesla V100 GPUs, each with 32 GB of memory, to accelerate the training and inference phases of the machine learning models.
- RAM: 128 GB DDR4, facilitating handling large datasets and computational tasks simultaneously.

- Storage: 1 TB SSD for quick access to the datasets and models.
- Operating System: Ubuntu 20.04 LTS, chosen for its stability and compatibility with machine learning libraries.
- Software and Libraries: TensorFlow 2.5 and PyTorch 1.8 were used as the primary framework for implementing the LSTM, CNN, and NLP models, along with other necessary libraries such as NumPy, Huggingface, pandas, and scikit-learn for data manipulation and analysis.
- Dataset: We use the dataset from IEEE Dataport, which contains 54K C programs. Of these 54K C programs, 15778 submissions are marked plagiarized, with plagiarism from level 1 to level 6. This distribution can be seen in Fig 10. We use 37800 C programs (containing 2240 files with level 1 labels, 2100 level 2, 1960 level 3, 1820 level 4, 1540 level 5, and 1384 level 6 plagiarism) for training, 8100 C programs for validation, and 8100 C programs for testing, containing 480 files with level 1 labels, 450 level 2, 420 level 3, 390 level 4, 330 level 5, and 297 level 6 plagiarism for validation and test sets each.

This configuration ensured the experiment's smooth execution, allowing for extensive model training and evaluation without significant bottlenecks in computational resources.

## 5.2 Evaluation Metrics and Computational Results

We understand that selecting appropriate evaluation metrics is vital for accurately gauging the efficacy of a plagiarism detection system. Our methodology, which leverages a combination of Convolutional Neural Networks (CNNs), Long Short-Term Memory networks (LSTMs), and Natural Language Processing (NLP) models for feature extraction, is designed to enhance the detection capabilities across various levels of plagiarism. Our model's performance was analyzed using several metrics

to illustrate the model's effectiveness and adaptability in plagiarism detection. Core metrics employed are:

- Precision and Recall are fundamental metrics that provide insights into the accuracy and completeness of the detection system, respectively. Precision is defined as the proportion of correctly identified plagiarized instances among all cases predicted as plagiarized, while recall is the proportion of actual plagiarized instances correctly identified. These metrics are particularly relevant in assessing the effectiveness of the plagiarism detection model:

$$\text{Precision} = \frac{TP}{TP + FP}$$
$$\text{Recall} = \frac{TP}{TP + FN}$$

- The F1-score offers a harmonic mean of precision and recall, providing a single metric to assess the balance between these two crucial aspects:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

  However, given the significant implications of failing to detect plagiarism, we also consider the F2-score, which weighs recall higher than precision, emphasizing the importance of minimizing false negatives. This metric is beneficial in highlighting our focus on detecting as many instances of plagiarism as possible:

$$\text{F2-score} = \frac{5 \times \text{Precision} \times \text{Recall}}{(4 \times \text{Precision}) + \text{Recall}}$$

  We calculate both the F1 and F2 scores to enable comparative analysis with existing research and to underscore our model's performance in terms of recall.

- The ROC AUC curve further complements these metrics by illustrating the trade-off between the true positive rate (recall) and the false positive rate at

various thresholds. Computed as below :

$$\text{AUC} = \int_{x=0}^{1} TPR(x)dx$$

Before discussing the detailed performance metrics, it's essential to understand the role of decision thresholds(obtained in Step 4 from Fig 4) in our plagiarism detection model. A decision threshold is a critical value that determines how the probabilities calculated by the model are converted into a binary outcome. We can adjust the model's sensitivity and specificity by varying this threshold to meet different operational needs. We evaluated our model across three thresholds(Fig 12), 0.3, 0.4, and

| Feature Vector | Threshold | Precision | Recall | F2-Score |
|---|---|---|---|---|
| Integrated | 0.3 | 0.85 | 0.93 | 0.91 |
| Integrated | 0.4 | 0.93 | 0.93 | 0.92 |
| Integrated | 0.5 | 0.94 | 0.87 | 0.88 |
| CNN only | 0.4 | 0.78 | 0.75 | 0.74 |
| LSTM only | 0.4 | 0.82 | 0.8 | 0.8 |
| NLP(CodePTM) only | 0.4 | 0.87 | 0.85 | 0.84 |

Fig. 12: Performance comparison of our proposed model with integrated and single-method feature vectors

0.5, to optimize for high recall rates, aiming to capture as many true positives as possible while maintaining an acceptable level of false positives. This approach allows us to adjust the model's sensitivity to identify plagiarism more effectively across various complexity levels. From the results, we realized that choosing a threshold of 0.3 could maximize recall, which is crucial when the cost of missing any instance of plagiarism is high. However, this comes at the expense of precision, increasing the number of false positives. This means more non-plagiarized submissions could be mistakenly flagged as plagiarized, leading to unnecessary reviews and potential disputes. Choosing a threshold of 0.5 typically results in a higher precision, effectively reducing the number of false positives. However, this increase in precision usually comes at the cost of lower recall. In the context of plagiarism detection, this means that while the cases identified as plagiarized are likely, the model may miss more instances of actual

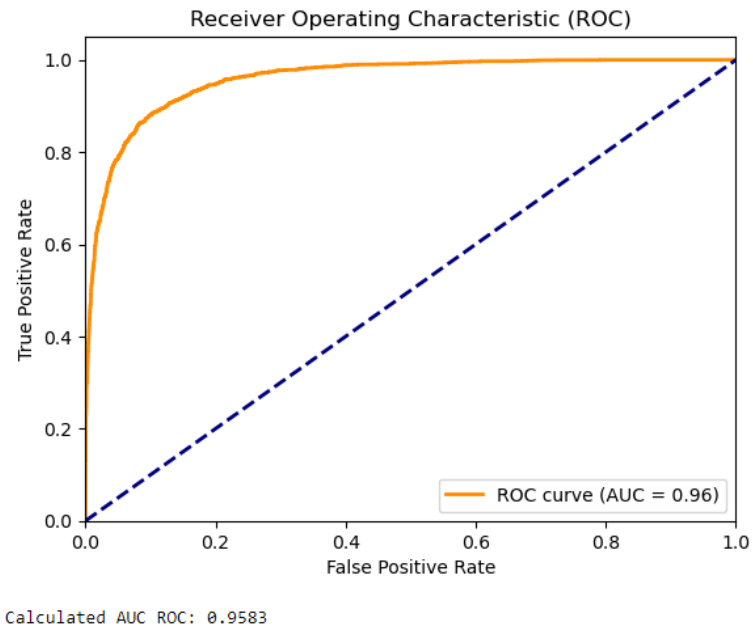


Fig. 13: AUC-ROC curve

plagiarism. In essence, at a threshold of 0.5, the model becomes more conservative, potentially overlooking subtler or more complex cases of plagiarism compared to a threshold of 0.4. This trade-off is not ideal when the goal is to ensure that most instances of plagiarism are detected, even if it results in some false positives. The model had a ROC-AUC score of 0.958(visualized in Fig 13), which highlights the model's ability to distinguish between plagiarized and non-plagiarized classes under different decision thresholds. Opting for a threshold of 0.4 is a compromise that still offers high recall but with better precision. This balance reduces the burden of false positives while effectively catching various plagiarism levels. This balanced approach often provides a more practical solution in educational settings, where false positives and negatives carry significant consequences. So, we decided to set our threshold to 0.4 for performance evaluation and comparison with other models and tools.

In this study, we demonstrate the efficacy of our integrated feature vector approach and scrutinize the performance of single-method models using only CNN, LSTM, or NLP. This analysis(shown in Fig 12) is crucial as it helps substantiate the hypothesis that combining multiple feature detection techniques enhances the model's accuracy

and robustness more effectively than using any single method alone. By presenting results from each technique separately, we aim to highlight the incremental benefits gained from the integration, thus providing a robust validation of our hybrid approach against more traditional, singular technique-based models. The model's performance was assessed using a singular threshold of 0.4(our optimal threshold) to ensure uniformity across comparisons. These performance metrics(shown in Fig 12) demonstrate that while each feature vector has its strengths, they often need to catch up in comprehensively detecting plagiarism across various levels and types. This highlights the necessity of our integrated feature vector approach, which combines the strengths of CNN, LSTM, and NLP to achieve better results, as discussed above and also seen in Fig 12. Following analyzing our model's performance at various thresholds, we've synthesized the results into a confusion matrix for a comprehensive overview. This matrix helps illustrate the model's performance in practical scenarios, mainly how it classifies plagiarized versus non-plagiarized submissions. For the chosen threshold of 0.4, the confusion matrix is shown in Fig 14. The confusion matrix provided in

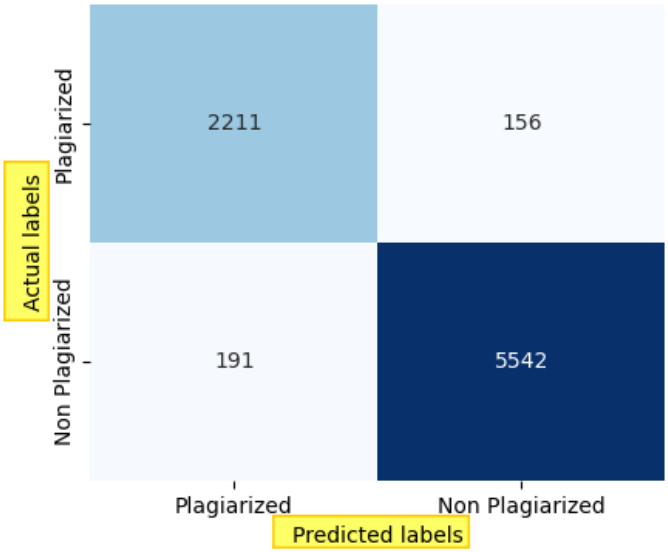


Fig. 14: Confusion matrix

Fig 14 illustrates the performance of our plagiarism detection model at a threshold of 0.4. The matrix reveals that the model successfully identified 2,211 out of 2,367 plagiarized cases, demonstrating a high True Positive rate. This performance under-

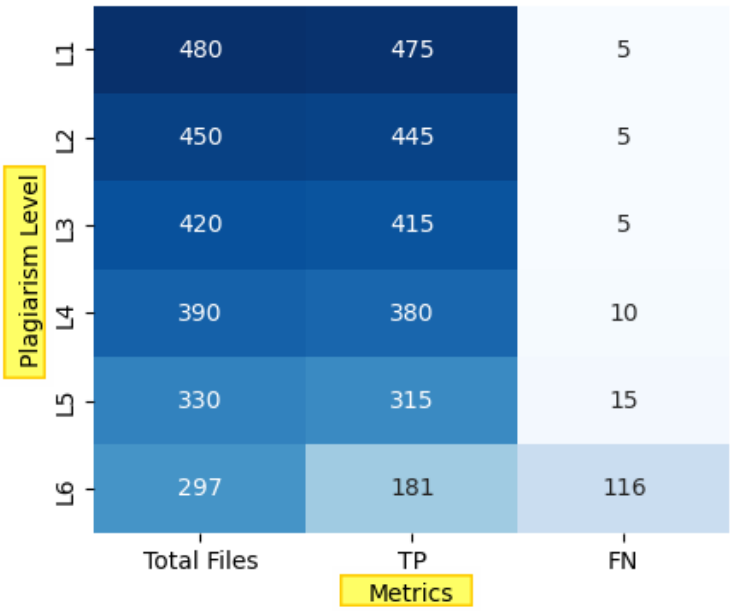


Fig. 15: Confusion matrix by levels

scores the model's high capability to detect actual instances of plagiarism effectively. However, the matrix also indicates that the model incorrectly flagged 191 out of 5,733 non-plagiarized cases as plagiarized. This occurrence illustrates the trade-offs between sensitivity (the ability to detect true positives) and specificity (the ability to avoid false positives) inherent in selecting threshold levels. The model demonstrates a strong capability to detect plagiarism, achieving a recall of 0.92 and a precision of 0.93, indicating high detection efficiency across these levels. Let us look at Fig 15, which shows how effectively our model distinguishes between various levels of plagiarism represented by levels L1 through L6 on the Y-axis. Here, the Total Files Column Indicates the total number of files evaluated per level, helping contextualize the True Positive(TP), False Positive(FP), and False Negative (FN) numbers. Each row in the matrix corresponds to the actual classes of plagiarism levels, while each column represents the predicted classifications made by the model. From Fig 15, the model successfully identified 475 cases in Level 1 (out of 480), 445 in Level 2 (out of 450), 415 in Level 3 (out of 420), 380 in Level 4 (out of 390), and 315 in Level 5 (out of 330). However, a critical area of concern highlighted by the matrix is the performance at Level 6, where the model detected only 181 out of 297 cases. This results in a detection

rate of approximately 61%, substantially lower than the other levels. This markedly lower detection rate suggests difficulties in accurately identifying the most sophisticated forms of plagiarism, often involving nuanced and design-level changes. The low detection rate at Level 6 reflects the complexity and subtlety of the plagiarism involved, which are more challenging to capture with the proposed methods. Advanced forms of plagiarism at this level often require deeper semantic and logical analysis, extending into areas such as design pattern recognition. Unlike lower levels of plagiarism, which may present more detectable patterns or similarities at the code surface level, Level 6 changes are less noticeable. Thus, existing detection methods based on token matching, syntax trees, or simple semantic models might not be effective. True Negatives are not included in the matrix(Fig 15) because the matrix categories (L1 through L6) are solely focused on levels of plagiarism. True Negatives would theoretically represent the non-plagiarized files, and since our classification levels are designed exclusively to categorize various degrees of plagiarism, non-plagiarized files do not fall into any of these predefined levels and thus are not represented in the matrix. From the results discussed above, we understand that, while demonstrating robust performance for less sophisticated plagiarism levels, the matrix directs our attention to the need for advanced techniques to better handle complex cases like those seen in Level 6. This analysis enhances our understanding of the model's capabilities and informs future enhancements to improve detection accuracy. Transitioning from our findings, we now compare our integrated approach's effectiveness at a threshold of 0.4 with existing tools and methodologies in the next section. This comparison highlights our model's significant enhancements in source code plagiarism detection, especially in detecting intricate and subtle cases that single-method systems might overlook. While our evaluation emphasizes recall and focuses on the "plagiarized" class, unlike other models that often target the majority "non-plagiarized" class, we will also present metrics(shown in Fig 16) consistent with standard practices to ensure our results are comparable. This dual approach allows us to underscore our model's strengths while facilitating a direct comparison with other studies. Some of the plagiarized code detected by our model can be seen in fig 16, 17, 18 and 19.

```cpp
class Matrix {
public:
    Matrix(int rows, int cols);
    void setElement(int row, int col, int value);
    int getElement(int row, int col) const;
    Matrix add(const Matrix& other) const;

private:
    int rows, cols;
    std::vector<std::vector<int>> data;
};

Matrix::Matrix(int rows, int cols) : rows(rows), cols(cols) {
    data.resize(rows, std::vector<int>(cols, 0));
}

void Matrix::setElement(int row, int col, int value) {
    data[row][col] = value;
}

int Matrix::getElement(int row, int col) const {
    return data[row][col];
}

Matrix Matrix::add(const Matrix& other) const {
    if (rows != other.rows || cols != other.cols) {
        throw std::invalid_argument("Matrix dimensions must agree.");
    }
    Matrix result(rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result.setElement(i, j, data[i][j] + other.getElement(i, j));
        }
    }
    return result;
}

int main() {
    Matrix matrix1(2, 2);
    Matrix matrix2(2, 2);
    matrix1.setElement(0, 0, 1);
    matrix1.setElement(0, 1, 2);
    matrix1.setElement(1, 0, 3);
    matrix1.setElement(1, 1, 4);

    matrix2.setElement(0, 0, 5);
    matrix2.setElement(0, 1, 6);
    matrix2.setElement(1, 0, 7);
    matrix2.setElement(1, 1, 8);

    Matrix resultAdd = matrix1.add(matrix2);

    std::cout << "Addition Result:\n";
    std::cout << resultAdd.getElement(0, 0) << " " << resultAdd.getElement(0, 1) << "\n";
    std::cout << resultAdd.getElement(1, 0) << " " << resultAdd.getElement(1, 1) << "\n";

    return 0;
}
```

```cpp
class Grid {
public:
    Grid(int r, int c);
    void updateValue(int r, int c, int val);
    int retrieveValue(int r, int c) const;
    Grid merge(const Grid& other) const;

private:
    int rows, cols;
    std::vector<std::vector<int>> matrixData;

    void checkDimensions(const Grid& other) const;
};

Grid::Grid(int r, int c) : rows(r), cols(c) {
    matrixData.resize(rows, std::vector<int>(cols, 0));
}

void Grid::updateValue(int r, int c, int val) {
    matrixData[r][c] = val;
}

int Grid::retrieveValue(int r, int c) const {
    return matrixData[r][c];
}

void Grid::checkDimensions(const Grid& other) const {
    if (rows != other.rows || cols != other.cols) {
        throw std::invalid_argument("Grid sizes must match.");
    }
}

Grid Grid::merge(const Grid& other) const {
    checkDimensions(other);
    Grid result(rows, cols);
    for (int i = 0; i < rows * cols; ++i) {
        int r = i / cols;
        int c = i % cols;
        result.updateValue(r, c, matrixData[r][c] + other.retrieveValue(r, c));
    }
    return result;
}

int main() {
    Grid gridA(2, 2);
    Grid gridB(2, 2);
    gridA.updateValue(0, 0, 1);
    gridA.updateValue(0, 1, 2);
    gridA.updateValue(1, 0, 3);
    gridA.updateValue(1, 1, 4);

    gridB.updateValue(0, 0, 5);
    gridB.updateValue(0, 1, 6);
    gridB.updateValue(1, 0, 7);
    gridB.updateValue(1, 1, 8);

    Grid resultSum = gridA.merge(gridB);

    std::cout << "Sum Result:\n";
    std::cout << resultSum.retrieveValue(0, 0) << " " << resultSum.retrieveValue(0, 1) << "\n";
    std::cout << resultSum.retrieveValue(1, 0) << " " << resultSum.retrieveValue(1, 1) << "\n";

    return 0;
}
```

Fig. 16: A case of structural and semantic obfuscation detected by our model

```cpp
class Grid {
public:
    Grid(int r, int c);
    void updateCell(int r, int c, int val);
    int fetchCell(int r, int c) const;
    Grid merge(const Grid& other) const;

private:
    int rows, cols;
    std::vector<std::vector<int>> matrixData;

    void checkDimensions(const Grid& other) const;
    void initialize(int rows, int cols);
};

Grid::Grid(int r, int c) {
    initialize(r, c);
}

void Grid::initialize(int r, int c) {
    rows = r;
    cols = c;
    matrixData.resize(rows, std::vector<int>(cols, 0));
}

void Grid::updateCell(int r, int c, int val) {
    matrixData[r][c] = val;
}
int Grid::fetchCell(int r, int c) const {
    return matrixData[r][c];
}
void Grid::checkDimensions(const Grid& other) const {
    if (rows != other.rows || cols != other.cols) {
        throw std::invalid_argument("Grid sizes must match.");
    }
}
Grid Grid::merge(const Grid& other) const {
    checkDimensions(other);
    Grid result(rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result.updateCell(i, j, matrixData[i][j] + other.fetchCell(i, j));
        }
    }
    return result;
}
int main() {
    Grid grid1(2, 2);
    Grid grid2(2, 2);
    grid1.updateCell(0, 0, 1);
    grid1.updateCell(0, 1, 2);
    grid1.updateCell(1, 0, 3);
    grid1.updateCell(1, 1, 4);

    grid2.updateCell(0, 0, 5);
    grid2.updateCell(0, 1, 6);
    grid2.updateCell(1, 0, 7);
    grid2.updateCell(1, 1, 8);

    Grid resultSum = grid1.merge(grid2);

    std::cout << "Sum Result:\n";
    std::cout << resultSum.fetchCell(0, 0) << " " << resultSum.fetchCell(0, 1) << "\n";
    std::cout << resultSum.fetchCell(1, 0) << " " << resultSum.fetchCell(1, 1) << "\n";
    return 0;
}
```

```cpp
class Matrix {
public:
    Matrix(int rows, int cols);
    void setElement(int row, int col, int value);
    int getElement(int row, int col) const;
    Matrix add(const Matrix& other) const;

private:
    int rows, cols;
    std::vector<std::vector<int>> data;
};

Matrix::Matrix(int rows, int cols) : rows(rows), cols(cols) {
    data.resize(rows, std::vector<int>(cols, 0));
}

void Matrix::setElement(int row, int col, int value) {
    data[row][col] = value;
}

int Matrix::getElement(int row, int col) const {
    return data[row][col];
}

Matrix Matrix::add(const Matrix& other) const {
    if (rows != other.rows || cols != other.cols) {
        throw std::invalid_argument("Matrix dimensions must agree.");
    }
    Matrix result(rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result.setElement(i, j, data[i][j] + other.getElement(i, j));
        }
    }
    return result;
}

int main() {
    Matrix matrix1(2, 2);
    Matrix matrix2(2, 2);
    matrix1.setElement(0, 0, 1);
    matrix1.setElement(0, 1, 2);
    matrix1.setElement(1, 0, 3);
    matrix1.setElement(1, 1, 4);

    matrix2.setElement(0, 0, 5);
    matrix2.setElement(0, 1, 6);
    matrix2.setElement(1, 0, 7);
    matrix2.setElement(1, 1, 8);

    Matrix resultAdd = matrix1.add(matrix2);

    std::cout << "Addition Result:\n";
    std::cout << resultAdd.getElement(0, 0) << " " << resultAdd.getElement(0, 1) << "\n";
    std::cout << resultAdd.getElement(1, 0) << " " << resultAdd.getElement(1, 1) << "\n";

    return 0;
}
```

Fig. 17: Control Structure and Logical Changes detected by our model

```cpp
class MatrixCalc {
public:
    MatrixCalc(int r, int c);
    void assignvalue(int r, int c, int val);
    int getvalue(int r, int c) const;
    MatrixCalc sum(const MatrixCalc& other) const;
    void printMatrix() const;
private:
    int rows, cols;
    std::vector<std::vector<int>> data;

    void verifyDimensions(const MatrixCalc& other) const;
    void initMatrix(int r, int c);
};
MatrixCalc::MatrixCalc(int r, int c) {
    initMatrix(r, c);
}
void MatrixCalc::initMatrix(int r, int c) {
    rows = r;
    cols = c;
    data.resize(rows, std::vector<int>(cols, 0));
}
void MatrixCalc::assignvalue(int r, int c, int val) {
    data[r][c] = val;
}
int MatrixCalc::getvalue(int r, int c) const {
    return data[r][c];
}
void MatrixCalc::verifyDimensions(const MatrixCalc& other) const {
    if (rows != other.rows || cols != other.cols) {
        throw std::invalid_argument("Matrix dimensions must agree.");
    }
}
MatrixCalc MatrixCalc::sum(const MatrixCalc& other) const {
    verifyDimensions(other);
    MatrixCalc result(rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result.assignvalue(i, j, data[i][j] + other.getvalue(i, j));
        }
    }
    return result;
}
void MatrixCalc::printMatrix() const {
    for (const auto& row : data) {
        for (int val : row) {
            std::cout << val << " ";
        }
        std::cout << std::endl;
    }
}
int main() {
    MatrixCalc matrixA(2, 2);
    MatrixCalc matrixB(2, 2);
    matrixA.assignvalue(0, 0, 1);
    matrixA.assignvalue(0, 1, 2);
    matrixA.assignvalue(1, 0, 3);
    matrixA.assignvalue(1, 1, 4);

    matrixB.assignvalue(0, 0, 5);
    matrixB.assignvalue(0, 1, 6);
    matrixB.assignvalue(1, 0, 7);
    matrixB.assignvalue(1, 1, 8);
    MatrixCalc resultSum = matrixA.sum(matrixB);
    std::cout << "Sum Result:\n";
    resultSum.printMatrix();
    return 0;
}
```

```cpp
class Matrix {
public:
    Matrix(int rows, int cols);
    void setElement(int row, int col, int value);
    int getElement(int row, int col) const;
    Matrix add(const Matrix& other) const;

private:
    int rows, cols;
    std::vector<std::vector<int>> data;
};

Matrix::Matrix(int rows, int cols) : rows(rows), cols(cols) {
    data.resize(rows, std::vector<int>(cols, 0));
}

void Matrix::setElement(int row, int col, int value) {
    data[row][col] = value;
}

int Matrix::getElement(int row, int col) const {
    return data[row][col];
}

Matrix Matrix::add(const Matrix& other) const {
    if (rows != other.rows || cols != other.cols) {
        throw std::invalid_argument("Matrix dimensions must agree.");
    }
    Matrix result(rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result.setElement(i, j, data[i][j] + other.getElement(i, j));
        }
    }
    return result;
}

int main() {
    Matrix matrix1(2, 2);
    Matrix matrix2(2, 2);
    matrix1.setElement(0, 0, 1);
    matrix1.setElement(0, 1, 2);
    matrix1.setElement(1, 0, 3);
    matrix1.setElement(1, 1, 4);

    matrix2.setElement(0, 0, 5);
    matrix2.setElement(0, 1, 6);
    matrix2.setElement(1, 0, 7);
    matrix2.setElement(1, 1, 8);

    Matrix resultAdd = matrix1.add(matrix2);

    std::cout << "Addition Result:\n";
    std::cout << resultAdd.getElement(0, 0) << " " << resultAdd.getElement(0, 1) << "\n";
    std::cout << resultAdd.getElement(1, 0) << " " << resultAdd.getElement(1, 1) << "\n";

    return 0;
}
```

Fig. 18: Case of Algorithmic Changes detected by our model

```cpp
class GridOperations {
public:
    GridOperations(int rowCount, int colCount);
    void setVal(int row, int col, int val);
    int getVal(int row, int col) const;
    GridOperations addGrids(const GridOperations& otherGrid) const;
    void displayGrid() const;
private:
    int rowCount, colCount;
    std::vector<std::vector<int>> gridData;
    void validateDimensions(const GridOperations& otherGrid) const;
    void initializeGrid(int rowCount, int colCount);
    int generateRandomValue() const;
};
GridOperations::GridOperations(int rowCount, int colCount) {
    initializeGrid(rowCount, colCount);
}
void GridOperations::initializeGrid(int rowCount, int colCount) {
    this->rowCount = rowCount;
    this->colCount = colCount;
    gridData.resize(rowCount, std::vector<int>(colCount, generateRandomValue()));
}
int GridOperations::generateRandomValue() const {
    return std::rand() % 100;
}
void GridOperations::setVal(int row, int col, int val) {
    gridData[row][col] = val;
}
int GridOperations::getVal(int row, int col) const {
    return gridData[row][col];
}
void GridOperations::validateDimensions(const GridOperations& otherGrid) const {
    if (rowCount != otherGrid.rowCount || colCount != otherGrid.colCount) {
        throw std::invalid_argument("Grid dimensions must match.");
    }
}
GridOperations GridOperations::addGrids(const GridOperations& otherGrid) const {
    validateDimensions(otherGrid);
    GridOperations resultGrid(rowCount, colCount);
    for (int i = 0; i < rowCount; ++i) {
        for (int j = 0; j < colCount; ++j) {
            resultGrid.setVal(i, j, gridData[i][j] + otherGrid.getVal(i, j));
        }
    }
    return resultGrid;
}
void GridOperations::displayGrid() const {
    for (const auto& row : gridData) {
        for (int val : row) {
            std::cout << val << " ";
        }
        std::cout << std::endl;
    }
}
int main() {
    GridOperations grid1(2, 2);
    GridOperations grid2(2, 2);
    grid1.setVal(0, 0, 1);
    grid1.setVal(0, 1, 2);
    grid1.setVal(1, 0, 3);
    grid1.setVal(1, 1, 4);
    grid2.setVal(0, 0, 5);
    grid2.setVal(0, 1, 6);
    grid2.setVal(1, 0, 7);
    grid2.setVal(1, 1, 8);
    GridOperations resultGrid = grid1.addGrids(grid2);
    std::cout << "Sum Result:\n";
    resultGrid.displayGrid();
    return 0;
}
```

```cpp
class Matrix {
public:
    Matrix(int rows, int cols);
    void setElement(int row, int col, int value);
    int getElement(int row, int col) const;
    Matrix add(const Matrix& other) const;

private:
    int rows, cols;
    std::vector<std::vector<int>> data;
};

Matrix::Matrix(int rows, int cols) : rows(rows), cols(cols) {
    data.resize(rows, std::vector<int>(cols, 0));
}

void Matrix::setElement(int row, int col, int value) {
    data[row][col] = value;
}

int Matrix::getElement(int row, int col) const {
    return data[row][col];
}

Matrix Matrix::add(const Matrix& other) const {
    if (rows != other.rows || cols != other.cols) {
        throw std::invalid_argument("Matrix dimensions must agree.");
    }
    Matrix result(rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result.setElement(i, j, data[i][j] + other.getElement(i, j));
        }
    }
    return result;
}

int main() {
    Matrix matrix1(2, 2);
    Matrix matrix2(2, 2);
    matrix1.setElement(0, 0, 1);
    matrix1.setElement(0, 1, 2);
    matrix1.setElement(1, 0, 3);
    matrix1.setElement(1, 1, 4);

    matrix2.setElement(0, 0, 5);
    matrix2.setElement(0, 1, 6);
    matrix2.setElement(1, 0, 7);
    matrix2.setElement(1, 1, 8);

    Matrix resultAdd = matrix1.add(matrix2);

    std::cout << "Addition Result:\n";
    std::cout << resultAdd.getElement(0, 0) << " " << resultAdd.getElement(0, 1) << "\n";
    std::cout << resultAdd.getElement(1, 0) << " " << resultAdd.getElement(1, 1) << "\n";

    return 0;
}
```

Fig. 19: Advanced Structural and Semantic Obfuscation case detected by our model

## 5.3 Comparing Results

The proposed solution has performed better in our comparative analysis(fig 20) of various plagiarism detection tools and models, particularly against established tools such as MOSS and JPlag, and research work is detailed in the literature review.

| Model/Tool | Accuracy | Precision | Recall | F1-Score | Detected Plagiarism Levels |
|---|---|---|---|---|---|
| Our Proposed Model(0.4 threshold) | 0.95 | 0.93 | 0.92 | 0.92 | Levels 1, 2, 3, 4, 5, some 6 |
| MOSS | 0.85 | 0.70 | 0.58 | 0.63 | Level 1 |
| JPlag | 0.87 | 0.75 | 0.82 | 0.78 | Level 1, 2 |
| Lit Review(MILEPOST GCC) [21] | 0.90 | 0.92 | 0.83 | 0.87 | Level 1, 2, some 3 |
| Lit Review(AST) [56] | 0.94 | 0.84 | 0.71 | 0.63 | Level 1, 2, 3 |
| Lit Review(CodePTM) [20] | 0.90 | 0.79 | 0.86 | 0.82 | Level 1, 2, 3, some 4 |

Fig. 20: Comparative analysis

Our Proposed Model excels across multiple levels of plagiarism, showing robust precision (0.93), recall (0.92), and an F1-score of 0.92 at threshold 0.4(from Fig 16), indicating its effective balance between accuracy and completeness. As already discussed, our methodology integrates multiple feature vectors that capture the code's textual, structural, and semantic aspects, enabling it to identify a broad range of plagiarism complexities, from simple text copying to syntactic alteration to structural modifications and sophisticated algorithm transformations. This can be observed in the confusion matrix shown in Fig 14. The decision threshold of 0.4 was optimized to balance sensitivity and specificity, allowing the model to identify subtle nuances across various plagiarism levels effectively. As discussed previously(in section 5.2), Our methodology fails to detect level 6 plagiarism efficiently, which involves detecting design-level changes. Continuing with our comparison with other tools, MOSS, a widely used tool, has limitations with a precision of 0.70, recall of 0.58, and an F1-score of 0.63, and it is primarily effective at detecting only the most straightforward cases (Level 1). JPlag offers decent results, especially in lower complexity scenarios (Levels 1 and 2), with a precision of 0.75, recall of 0.82, and an F1-score of 0.78. Further, comparing our results with literature review works that used the same dataset [21], [56], shows that research work [21] that used MILEPOST GCC is comparable to our model in precision (0.92) but falls short in recall (0.83) and F1-score (0.87) and

can detect up to Level 3 plagiarism. The research work [56] used AST-based methods and demonstrated moderate effectiveness (Precision: 0.84, Recall: 0.71, F1-Score: 0.63) across Levels 1 to 3. The research work [20], which solely used CodePTM, can detect some of the level 4 files, too (Precision: 0.79, Recall: 0.86, F1-Score: 0.82). The levels of plagiarism identified by each research work indicated in Fig 16(rows 4, 5, and 6) are obtained directly from the published results. These comparisons underscore the capability of our model to detect a broader range of plagiarism types more accurately, reinforcing the benefits of integrating multiple feature extraction techniques to enhance detection accuracy across various plagiarism levels.

In our comparative analysis in Fig 20, we utilized published results from literature review papers to assess the performance metrics of various plagiarism detection algorithms detailed within those studies. For tools such as MOSS and JPlag, we were not able to get the information on the dataset used. We relied on performance results as documented in prior research studies [21], [48], [55]. With the completion of the experiments and analysis in Chapter 5, our results strongly support the efficacy of employing multiple feature vectors(generated by CNN, LSTM, and CodePTM) in accurately detecting code plagiarism. However, we recognize that there are opportunities for improvement and learning. As we wrap up this thesis in Chapter 6, we will leverage the insights obtained to contemplate future enhancements that could further refine plagiarism detection technologies. This final chapter will focus on the implications of our findings and explore new avenues in machine learning and academic integrity.

# CHAPTER 6

# *Conclusion and Future Work*

As we conclude this thesis, we revisit the objectives we initially set out to achieve and evaluate our results. In this research, we developed a plagiarism detection system that employs a sophisticated feature vector combining syntactic, structural, and semantic analyses through convolutional neural networks (CNNs), recurrent neural networks (RNNs), and natural language processing (NLP) techniques, referred to here as CodePTMs. This unique combination, not found in the existing literature, significantly enhances the detection of nuanced forms of plagiarism, including sophisticated disguises not typically uncovered by conventional methods.

Our approach incorporates and utilizes a novel amalgamation of tokenization, abstract syntax tree (AST) generation, and metadata analysis to enrich the feature set further, distinguishing our work from other plagiarism detection techniques that often overlook these elements. The system has demonstrated high effectiveness, achieving an accuracy of 0.95 in identifying plagiarism across various complexity levels within C code submissions. Specifically, these performance metrics were calculated by focusing on the plagiarized class, indicating a balanced capability to identify true positives without a high rate of false negatives. This focused assessment underscores the potential of our integrated feature extraction strategy to advance the field of plagiarism detection significantly, particularly in identifying and handling instances of plagiarism effectively.

This research has successfully introduced a source code plagiarism detection sys-

tem; however, its practical implementation faces certain limitations. Primarily, the computational intensity of our methods, which involve CNNs, RNNs, and advanced NLP techniques for detailed feature extraction, means that the system is best suited for smaller, course-specific datasets rather than broad applications across an entire university's database. This limitation is due to resource constraints, as the extensive data processing, feature extraction, and training required for accurate plagiarism detection can be resource-intensive.

## 6.1 Propositions for Future Endeavors

Looking ahead, the potential for further advancements in plagiarism detection remains vast. Future research could explore several promising directions to enhance the scalability and adaptability of our model. One such avenue is extending the model's applicability across different programming languages, which would broaden its utility and impact. Additionally, incorporating unsupervised learning techniques could improve the model's adaptability, enabling it to learn from unlabeled data and potentially uncover new patterns of plagiarism autonomously.

Real-time plagiarism detection in educational assignment submission platforms is another critical area for development. This would allow for immediate feedback and deterrence, proactively enhancing academic integrity. Furthermore, exploring the application of transfer learning could allow models trained on one set of programming languages to be effectively adapted to others, thereby reducing the need for extensive retraining.

Emerging machine learning techniques also present exciting opportunities for advancing plagiarism detection. Generative Adversarial Networks (GANs) could be utilized to create synthetic examples of plagiarism, providing a rich dataset for training without the ethical and legal complications of using actual student code. Similarly, reinforcement learning could be employed to continuously refine the model's decision-

making process based on feedback, leading to progressively more accurate detection over time.

## 6.2 Concluding Reflections

In conclusion, this research represents a collaborative achievement for which I am deeply grateful. Special thanks are owed to my thesis advisor, Dr. Wu, whose invaluable guidance and expertise have profoundly shaped this work. Additionally, I sincerely appreciate the members of my thesis committee whose insightful feedback has been instrumental in refining our approaches and conclusions.

This thesis does not represent an endpoint but rather a gateway to future scholarly exploration. It is with great anticipation that we look forward to the contributions this work will inspire in machine learning and academic integrity.

# REFERENCES


[1] Agrawal, M. and Sharma, D. (2016). A state of art on source code plagiarism detection. In *2016 2nd International Conference on NGCT*, pages 236–241.

[2] Aiken, A. (2004). Moss: A system for detecting software plagiarism. http://www.cs.berkeley.edu/ aiken/moss.

[3] Akbulut, Y. and Dönmez, O. (2018). Predictors of digital piracy among turkish undergraduate students. *Telematics Inform*, 35(5):1324–1334.

[4] Ali, A. M. E. T., Abdulla, H. M. D., and Snasel, V. (2011). Overview and comparison of plagiarism detection tools. In *Dateso*, pages 161–172.

[5] Aniceto, R. C., Holanda, M., Castanho, C., and Da Silva, D. (2021). Source code plagiarism detection in an educational context: A literature mapping. In *2021 IEEE FIE*, pages 1–9. IEEE.

[6] Author(s) (2020). Plagiarism detection in programming assignments using machine learning. *Journal of Artificial Intelligence and Capsule Networks*, 2(3):177–184.

[7] Bandara, U. and Wijayrathna, G. (2012). Detection of source code plagiarism using machine learning approach. *Int J Comput Theory Eng*, 4(5):674–678.

[8] Bayaga, A. (2010). Multinomial logistic regression: usage and application in risk analysis. *J Appl Quant Methods*, 5(2):288–297.

[9] Bharti, K. and Singh, P. (2015). Hybrid dimension reduction by integrating feature selection with feature extraction method for text clustering. *Expert Syst Appl*, 42(6):3105–3114.

[10] Buddrus, F. and Schödel, J. (1998). Cappuccino—a c++ to java translator. In *ACM Symposium on Applied Computing (SAC)*, Atlanta, GA.

[11] Chae, D.-K., Ha, J., Kim, S.-W., Kang, B., and Im, E. (2013a). Software plagiarism detection: a graph-based approach. In *22nd ACM International Conference on Information and Knowledge Management*, San Francisco, CA.

[12] Chae, D.-K., Kim, S.-W., Ha, J., Lee, S.-C., and Woo, G. (2013b). Software plagiarism detection via the static api call frequency birthmark. In *28th Annual ACM Symposium on Applied Computing (SAC)*, Coimbra, Portugal.

[13] Cheers, H., Lin, Y., and Smith, S. P. (2019). A novel approach for detecting logic similarity in plagiarised source code. In *2019 IEEE ICSESS*, pages 1–6. IEEE.

[14] Cheers, H., Lin, Y., and Smith, S. P. (2020). Detecting pervasive source code plagiarism through dynamic program behaviours. In *Proceedings of the Twenty-Second Australasian Computing Education Conference*, pages 21–30.

[15] Cheers, H., Lin, Y., and Smith, S. P. (2021). Academic source code plagiarism detection by measuring program behavioral similarity. *IEEE Access*, 9:50391–50412.

[16] Choi, S., Park, H., Lim, H., and Han, T. (2009). A static api birthmark for windows binary executables. *J Syst Softw*, 82(5):862–873.

[17] Cosma, G. and Joy, M. (2012). An approach to source-code plagiarism detection and investigation using latent semantic analysis. *IEEE Trans Comput*, 61(3):379–394.

[18] Cˆımpeanu, A.-C. and Olteanu, A. (2019). Material survey on source code plagiarism detection in programming courses.

[19] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

[20] Ebrahim, F. and Joy, M. (2023). Source code plagiarism detection with pre-trained model embeddings and automated machine learning. In *Proceedings of the 14th International Conference on Recent Advances in Natural Language Processing*, pages 301–309.

[21] Eppa, A. and Murali, A. (2022). Source code plagiarism detection: A machine intelligence approach. In *2022 IEEE Fourth International Conference on Advances in Electronics, Computers and Communications (ICAECC)*, pages 1–7. IEEE.

[22] Estgren, M. (2018). Linux kernel code - on demand.

[23] Faidhi, J. A. and Robinson, S. K. (1987). An empirical approach for detecting program similarity and plagiarism within a university programming environment. *Computers & Education*, 11(1):11–19.

[24] Farhan, M., Jabbar, S., and Aslam, M. (2018). Iot-based students interaction framework using attention-scoring assessment in elearning. *IEEE Trans Softw Eng*, 79:909–919.

[25] Farhan, M., Jabbar, S., Aslam, M., et al. (2017). A real-time data mining approach for interaction analytics assessment: Iot based student interaction framework. *Int J Parallel Program*, 79:909–919. https://doi.org/10.1007/s10766-017-0553-7.

[26] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory.

[27] Husain, H., Wu, H.-H., Gazit, T., Allamanis, M., and Brockschmidt, M. (2019). Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*.

[28] Imran, S., Khan, M., Idrees, M., Muneer, I., and Iqbal, M. (2018). An enhanced framework for extrinsic plagiarism avoidance for research article. *Tech J*, 23(1):90–97.

[29] Jhi, Y.-C., Jia, X., Wang, X., Zhu, S., Liu, P., and Wu, D. (2015). Program characterization using runtime values and its application to software plagiarism detection. *IEEE Trans Softw Eng*, 41(9):925–943.

[30] Joy, M. and Luck, M. (1999). Plagiarism in programming assignments. *IEEE Transactions on Education*, 42(2):129–133.

[31] Karpathy, A. (2015). The unreasonable effectiveness of recurrent neural networks.

[32] Karpathy, A., Johnson, J., and Fei-Fei, L. (2015). Visualizing and understanding recurrent networks.

[33] Kashyap, V., Brown, D., Liblit, B., Melski, D., and Reps, T. (2017). Source forager: a search engine for similar source code. Technical report, arXiv preprint arXiv:1706.02769.

[34] Kustanto, C. and Liem, I. (2009). Automatic source code plagiarism detection. In *2009 10th ACIS International Conference on SNPD*, pages 481–486. IEEE.

[35] Lazar, F.-M. and Banias, O. (2014). Clone detection algorithm based on the abstract syntax tree approach. In *9th International Symposium on Applied Computational Intelligence and Informatics (SACI)*, Timisoara, Romania.

[36] Li, J., Bioucas-Dias, J., and Plaza, A. (2010). Semisupervised hyperspectral image segmentation using multinomial logistic regression with active learning. *IEEE Trans Geosci Remote Sens*, 48(11):4085–4098.

[37] Lim, H.-I., Park, H., Choi, S., and Han, T. (2009). A method for detecting the theft of java programs through analysis of the control flow information. *Inf Softw Technol*, 51(9):1338–1350.

[38] Liu, C., Chen, C., Han, J., and Yu, P. (2006). Gplag: detection of software plagiarism by program dependence graph analysis. In *12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, Philadelphia, PA.

[39] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

[40] Luo, L., Ming, J., Wu, D., Liu, P., and Zhu, S. (2017). Semantics-based obfuscation-resilient binary code similarity comparison with applications to software and algorithm plagiarism detection. *IEEE Trans Softw Eng*, 43(12):1157–1177.

[41] Mansournia, M., Geroldinger, A., Greenland, S., and Heinze, G. (2017). Separation in logistic regression–causes, consequences, and control. *Am J Epidemiol*, 187(4):864–870.

[42] Mišić, M. J., Protić, J. , and Tomašević, M. V. (2017). Improving source code plagiarism detection: lessons learned. In *2017 25th TELFOR*, pages 1–8.

[43] Ohno, A. and Murao, H. (2011). A two-step in-class source code plagiarism detection method utilizing improved cmalgorithm and sim. *Int J Innov Comput Inf Control*, 7(8):4729–4739.

[44] Park, H., Choi, S., Lim, H., and Han, T. (2008). Detecting java theft based on static api trace birthmark. In *International Workshop on Security (IWSEC)*, Kagawa, Japan.

[45] Platt, J. (2000). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Adv. Large Margin Classif.*, 10.

[46] Prado, B., Bispo, K. A., and Andrade, R. (2018). X9: An obfuscation resilient approach for source code plagiarism detection in virtual learning environments. In *ICEIS (1)*, pages 517–524.

[47] Queirós, R. and Leal, J. (2012). Petcha: a programming exercises teaching assistant. In *17th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE)*, Haifa, Israel.

[48] Ragkhitwetsagul, C. (2016). Measuring code similarity in large-scaled code corpora. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Raleigh, NC.

[49] Roy, C. and Cordy, J. (2007). A survey on software clone detection research. Technical Report 541, Queen's Sch Comput TR.

[50] Roy, S. K. and Deb, R. P. (2021). *Source Code Plagiarism Detection: A Machine Intelligence Approach*. Publisher Name (if available).

[51] Shanmughasundaram, M. and Subramani, S. (2015). A measurement of similarity to identify identical code clones. *Int Arab Inf Technol (IAJIT)*, 12:735–740.

[52] Shlens, J. (2014). A tutorial on principal component analysis. arXiv preprint arXiv:1404.1100.

[53] Smith, L. (2002). A tutorial on principal components analysis. arXiv preprint arXiv:1404.1100.

[54] Son, J.-W., Noh, T.-G., Song, H.-J., and Park, S.-B. (2013). An application for plagiarized source code detection based on a parse tree kernel. *Eng Appl Artif Intel*, 26(8):1911–1918.

[55] Son, J.-W., Park, S.-B., and Park, S.-Y. (2006). Program plagiarism detection using parse tree kernels. In *9th Pacific Rim International Conference on Artificial Intelligence (PRICAI)*, Guilin, China.

[56] Song, H.-J., Park, S.-B., and Park, S. (2015). Computation of program source code similarity by composition of parse tree and call graph. *Math Probl Eng*.

[57] Suttichaya, V., Eakvorachai, N., and Lurkraisit, T. (2022). Source code plagiarism detection based on abstract syntax tree fingerprintings. In *2022 17th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, pages 1–6. IEEE.

[58] Ullah, F., Wang, J., Farhan, M., Jabbar, S., Wu, Z., and Khalid, S. (2018). Plagiarism detection in students' programming assignments based on semantics: multimedia e-learning based smart assessment methodology. *Multimed Tools Appl*, pages 1–18. https://doi.org/10.1007/s11042-018-5827-6.

[59] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, , and Polosukhin, I. (2017). Attention is all you need.

[60] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., and Brew, J. (2019). Huggingface's transformers: State-of-the-art natural language processing. *arXiv*, 18:arXiv–1910.

[61] Wu, P., Wang, J., and Tian, B. (2018). Software homology detection with software motifs based on function-call graph. *IEEE Access*, 6:19007–19017.

[62] Yasaswi, J., Kailash, S., Chilupuri, A., Purini, S., and Jawahar, C. (2017). Unsupervised learning based approach for plagiarism detection in programming assignments. In *10th Innovations in Software Engineering Conference (ISEC)*, Jaipur, India.

[63] Zhang, F., Li, L., Liu, C., and Zeng, Q. (2020). Flow chart generation-based source code similarity detection using process mining. *Scientific Programming*.

[64] Zhang, F., Wu, D., Liu, P., and Zhu, S. (2014). Program logic based software plagiarism detection. In *25th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, Naples, Italy.

# VITA AUCTORIS

| | |
|---|---|
| NAME: | Sumisha Surendran |
| EDUCATION: | Master of Science in Computer Science(AI Stream), University of Windsor, Canada |
| | Bachelor of Technology in Computer Science, Jawaharlal Nehru Technological University, India |
| | Bachelor of Science in Applied Mathematics, University of Colorado Boulder, USA |